

Licenciatura en Informática

Universidad Siglo 21 — Modalidad a distancia

Seminario de Práctica — Módulo 3

SistRUGBY-SLTC

Sistema de Gestión de Partidos y Estadísticas de Rugby

Santiago Lawn Tennis Club — Santiago del Estero

Trabajo Práctico N° 3

AP3 — Implementación del prototipo en Java (POO)

Entrega incremental sobre AP1 y AP2

Alumno: Fonzo, Ignacio

Fecha: Junio de 2026

Repositorio: <https://github.com/nachof9/SistRUGBY-SLTC>

FONZO-IGNACIO-AP3.PDF

1. Introducción

El presente documento corresponde al Trabajo Práctico N° 3 (AP3) de la materia Seminario de Práctica de la Licenciatura en Informática de la Universidad Siglo 21, modalidad a distancia. Constituye una entrega incremental que retoma el análisis y diseño definidos en AP1 y AP2 e incorpora la implementación del prototipo en Java aplicando los pilares de la programación orientada a objetos.

Atendiendo las observaciones del docente sobre la entrega anterior — donde se solicitó incluir el contenido del trabajo previo dado el carácter iterativo e incremental del PUD —, este documento integra los apartados de AP1 y AP2 actualizados, y agrega la nueva sección AP3 con la codificación del prototipo, la trazabilidad de casos de uso seleccionados desde el diseño hasta las pruebas, las estructuras de datos y los algoritmos de ordenación y búsqueda aplicados.

El sistema SistRUGBY-SLTC tiene como objetivo centralizar la gestión operativa de la sección de rugby del Santiago Lawn Tennis Club (SLTC) de Santiago del Estero, reemplazando los registros manuales y planillas no integradas que se utilizan actualmente, y proveyendo al cuerpo técnico de información oportuna y confiable para la toma de decisiones.

2. Resumen de AP1 — Análisis y definición del proyecto

2.1 Justificación

La sección de rugby del SLTC enfrenta una problemática concreta: la ausencia de un sistema centralizado que permita gestionar jugadores, registrar partidos y generar estadísticas de manera sistemática. El cuerpo técnico y el secretario deportivo registran los datos en papel o planillas Excel individuales sin integración. Los problemas identificados son: pérdida o inconsistencia de datos históricos, imposibilidad de generar reportes ágiles, dificultad de seguimiento de rendimiento individual, tiempo administrativo excesivo, y riesgo de pérdida por uso de soportes físicos sin respaldo.

El desarrollo de SistRUGBY-SLTC permitirá centralizar el padrón de jugadores con historial deportivo completo, registrar sistemáticamente los partidos con sus eventos detallados (tries, tarjetas, sustituciones), generar estadísticas automáticas individuales y grupales por temporada, ofrecer reportes accesibles en tiempo real al cuerpo técnico y mantener una base de datos persistente que reemplace los registros en papel.

2.2 Alcance del sistema

El sistema cubre las divisiones juveniles M15, M16, M17, M18 y M19, y el plantel superior (Pre-Intermedia, Intermedia y Primera). Quedan fuera del alcance las categorías infantiles, la gestión financiera, la transmisión en vivo, la integración con redes sociales y otras disciplinas del club. El prototipo es una aplicación de escritorio con persistencia en MySQL; no incluye versión web ni móvil.

2.3 Requerimientos funcionales y no funcionales

Atendiendo la observación del docente sobre la asignación explícita de actores en cada requerimiento funcional, se reformula la tabla original. A continuación se presentan los requerimientos vigentes:

ID	Actor responsable	Descripción	Prioridad
RF01	Secretario	Registrar, modificar, dar de baja y consultar jugadores con datos personales, posición y categoría.	Alta
RF02	Entrenador	Registrar partidos indicando fecha, equipo rival, resultado final, sede y categoría.	Alta
RF03	Entrenador	Asociar jugadores a un partido, indicando titular o suplente.	Alta
RF04	Entrenador	Registrar eventos durante el partido: try, conversión, penal, drop, tarjetas y sustitución.	Alta
RF05	Entrenador	Generar estadísticas por jugador: partidos, tries, conversiones, penales, tarjetas y puntos aportados.	Alta
RF06	Entrenador/ Secretario	Generar reporte por partido con datos del encuentro, plantel y eventos.	Media
RF07	Administrador	Administrar las divisiones (M15..M19, Pre-Intermedia, Intermedia, Primera).	Media
RF08	Secretario	Registrar y mantener el listado de clubes rivales.	Media
RF09	Todos los roles	Acceso al sistema mediante usuario y contraseña con roles diferenciados.	Alta
RF10	Sistema	Organizar partidos y estadísticas por temporada (año).	Media

ID	Categoría	Descripción
RNF01	Seguridad	Acceso protegido por autenticación. Contraseñas almacenadas con hash robusto (PBKDF2 o bcrypt).
RNF02	Rendimiento	Consultas de estadísticas y reportes en menos de 3 segundos para datasets de hasta 500 partidos.
RNF03	Usabilidad	Interfaz intuitiva; no requiere capacitación técnica especializada para usuarios finales.
RNF04	Confiabilidad	Integridad referencial garantizada por restricciones de clave foránea.
RNF05	Portabilidad	Ejecutable en Windows 10/11 con JRE 17+.
RNF06	Mantenibilidad	Código siguiendo POO, organizado en capas (presentación, negocio, persistencia).
RNF07	Disponibilidad	Disponible en horario operativo del club (7:00–23:00 hs).

2.4 Procesos de negocio actuales (As-Is)

Se identifican tres procesos críticos en la operación actual de la sección rugby del SLTC, todos hoy ejecutados manualmente: inscripción de jugadores, organización y registro de partidos, y seguimiento estadístico. Los diagramas de actividad se mantienen del AP1 original.

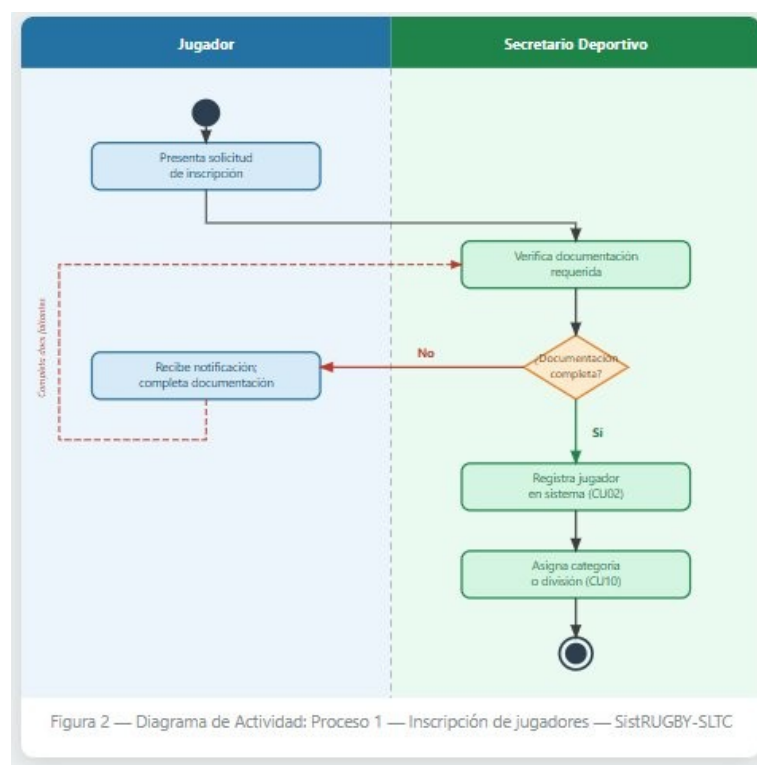


Figura 1 — Diagrama de Actividad: Proceso de inscripción de jugadores

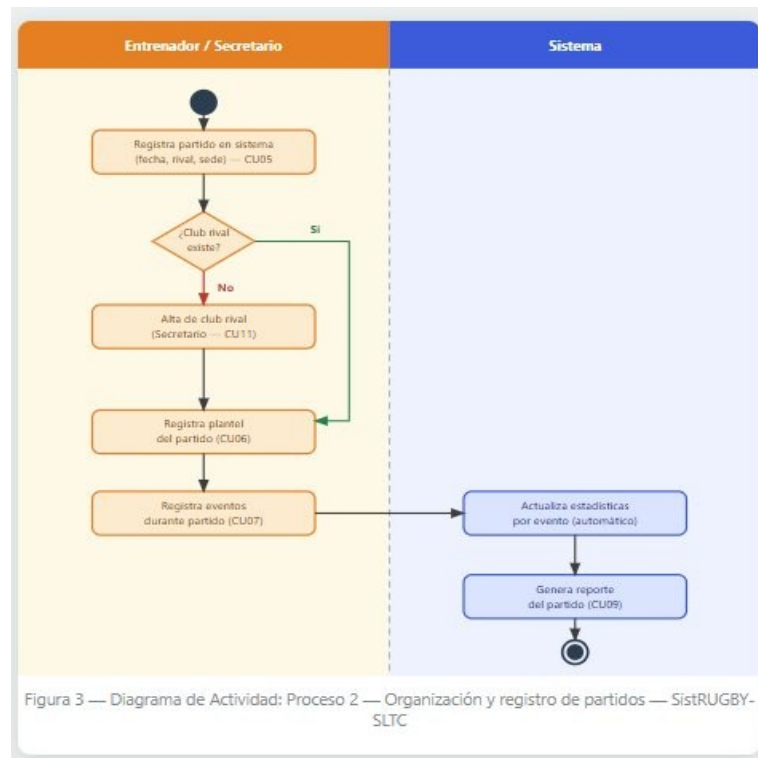


Figura 2 — Diagrama de Actividad: Organización y registro de partidos

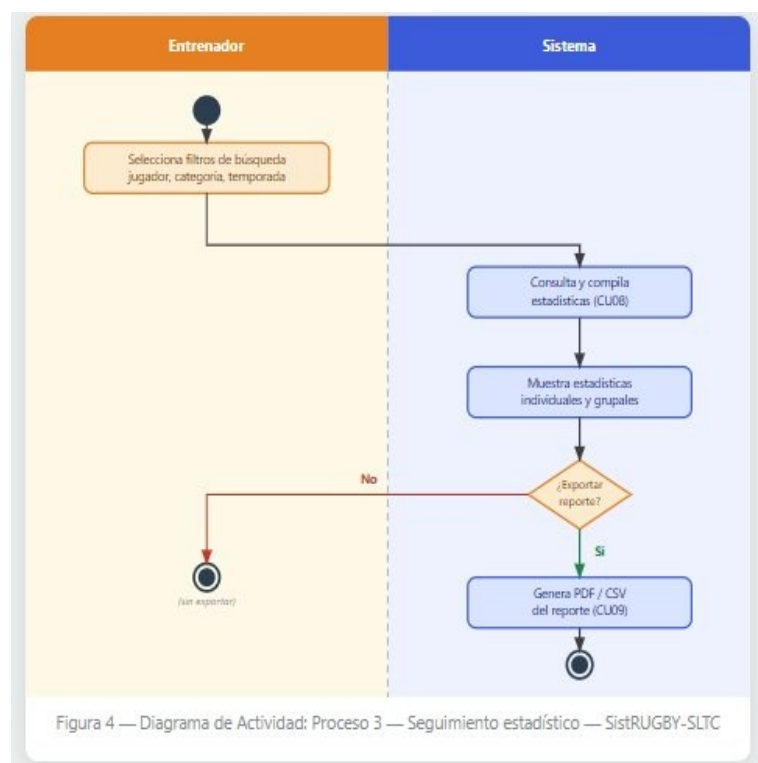


Figura 3 — Diagrama de Actividad: Seguimiento estadístico

2.5 Casos de uso

Se definieron doce casos de uso organizados en cuatro grupos funcionales: Gestión de Jugadores (CU02-CU04), Gestión de Partidos (CU05-CU07), Estadísticas y Reportes (CU08-CU09) y Configuración del Sistema (CU10-CU12). Todos los actores participan en CU01 (Iniciar sesión).

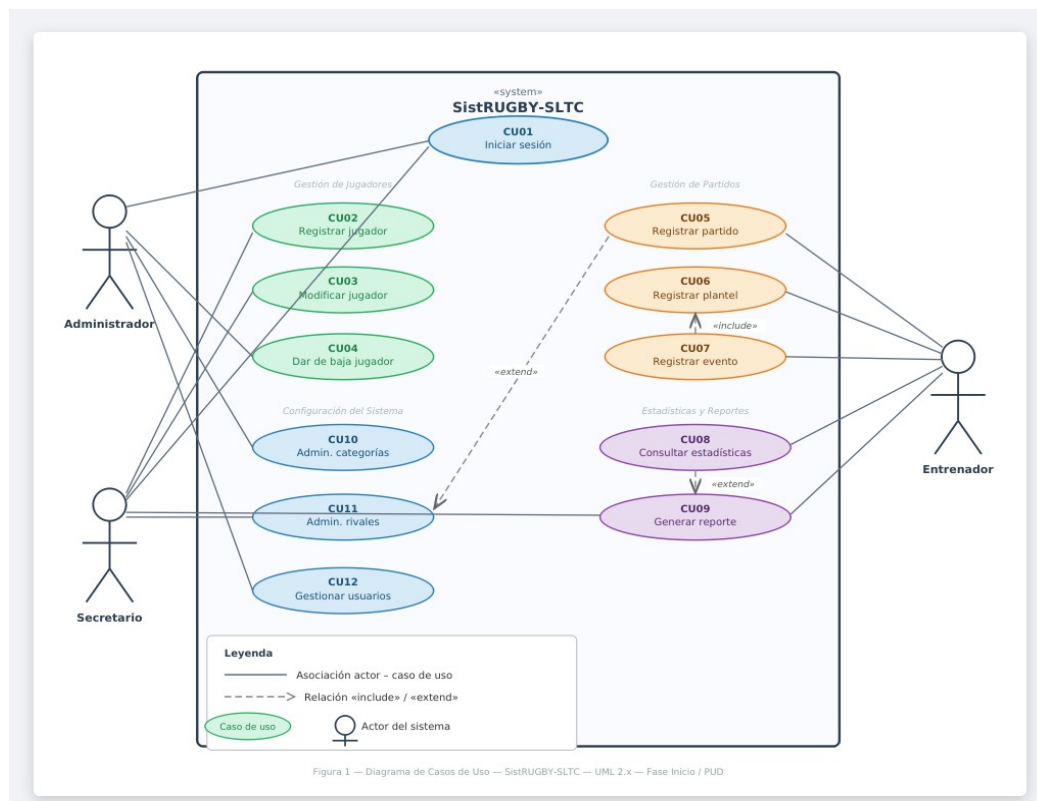


Figura 4 — Diagrama de Casos de Uso — UML 2.x — Fase Inicio del PUD

La trazabilidad y descripción narrativa detallada de los doce casos de uso (precondiciones, flujo principal, flujo alternativo, postcondición) se conserva del AP1; la sección 5 del presente documento desarrolla la trazabilidad completa desde el caso de uso hasta el código y las pruebas para tres CU representativos (CU01, CU02 y CU07).

2.6 Análisis de riesgos

En el marco del PUD, la fase de Inicio requiere la identificación y evaluación de los principales riesgos del proyecto. Se mantienen los diez riesgos identificados en AP1, destacándose como riesgos críticos: la modificación de requisitos durante la construcción (mitigado mediante congelamiento de alcance en Elaboración), el carácter unipersonal del proyecto (mitigado con documentación técnica actualizada en GitHub e hitos semanales), y la inconsistencia de la información en planillas Excel originales (mitigado mediante auditoría y reglas de limpieza previas a la migración).

3. Resumen de AP2 — Análisis profundo, diseño y base de datos

3.1 Etapa de análisis — Fase de Elaboración del PUD

La Fase de Elaboración tiene como objetivo estabilizar la arquitectura del sistema, refinar los casos de uso de mayor riesgo e identificar los elementos del dominio. Los artefactos principales son el modelo de dominio, los casos de uso detallados y los diagramas de interacción.

3.1.1 Diagrama de dominio

El diagrama de dominio identifica las ocho entidades conceptuales del negocio: Jugador (actor central), Partido (evento que aglutina plantel y eventos), EventoPartido (try, conversión, penal, drop, tarjeta, sustitución), PlantelPartido (relación ternaria jugador-partido-condición), Club (rival registrado), Categoría (división juvenil o plantel superior), Temporada y Usuario.

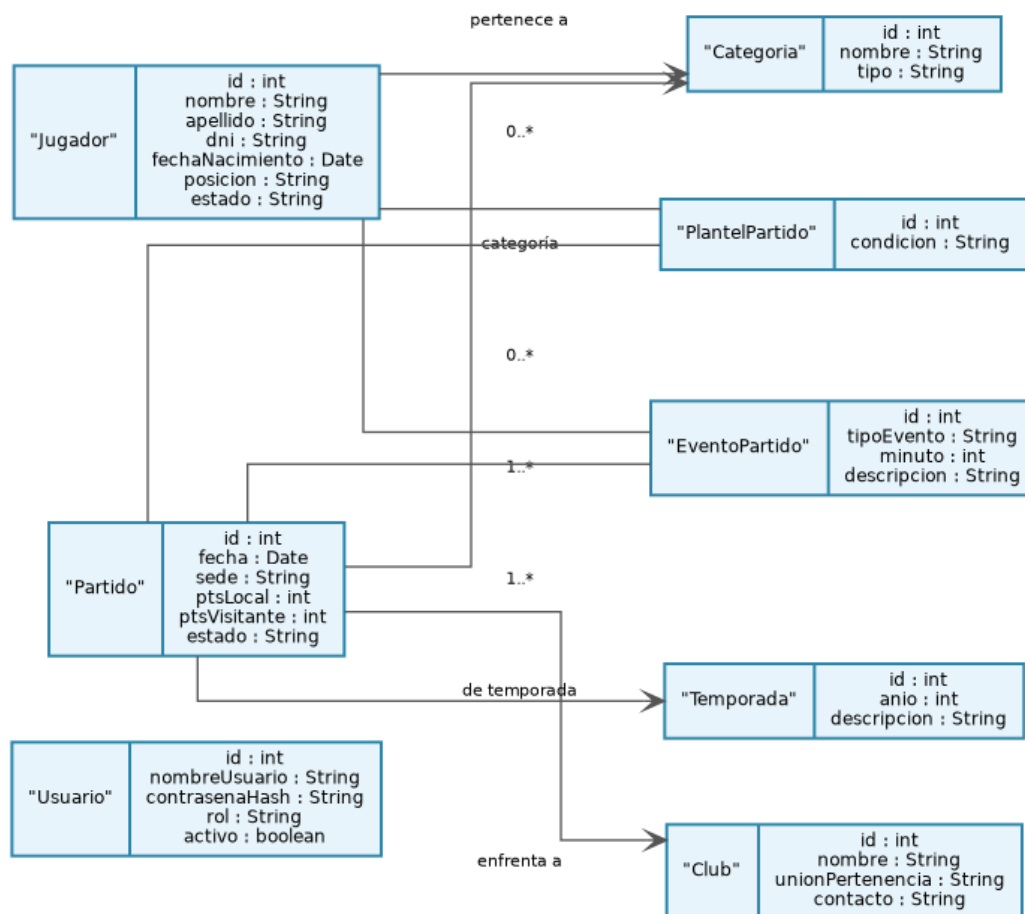


Figura 5 — Diagrama de Dominio — UML 2.x

3.1.2 Diagrama de casos de uso (revisado)

Atendiendo la corrección del docente, los nombres del diagrama coinciden exactamente con los de la tabla de casos de uso. Se mantienen los doce CU organizados en cuatro grupos funcionales con sus relaciones de include y extend.

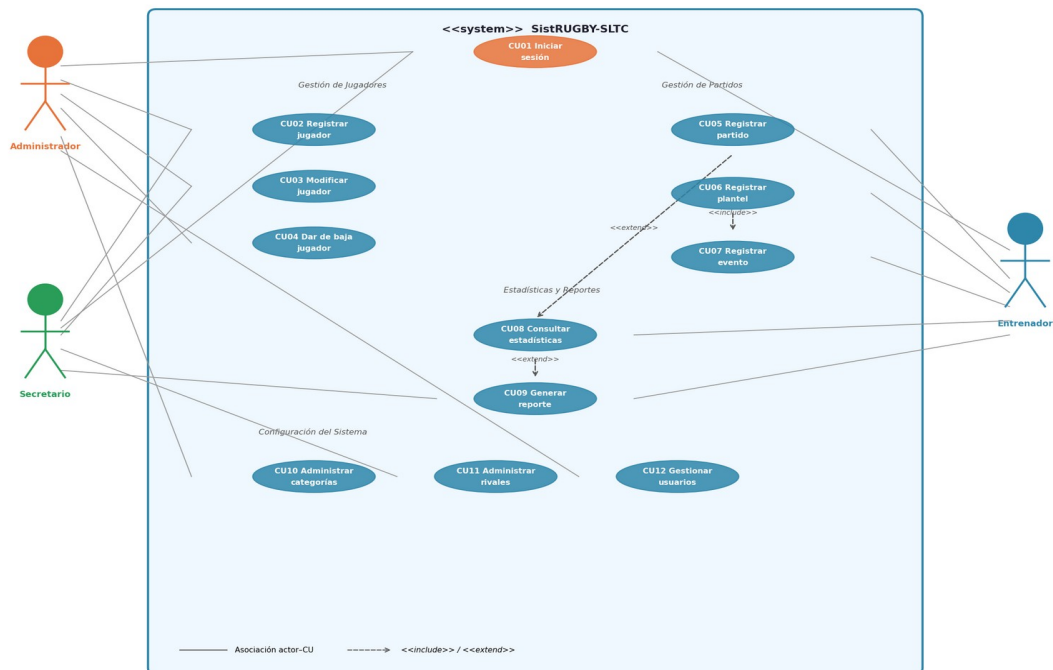


Figura 6 — Diagrama de Casos de Uso — UML 2.x — Fase Elaboración

3.1.3 Diagramas de secuencia

Se elaboraron diagramas de secuencia para los tres casos de uso de mayor criticidad arquitectónica: CU01 (autenticación), CU05 (registro de partido) y CU07 (registro de evento). Estos diagramas muestran la colaboración entre las tres capas — presentación, negocio y persistencia — y la interacción con MySQL a través de JDBC.

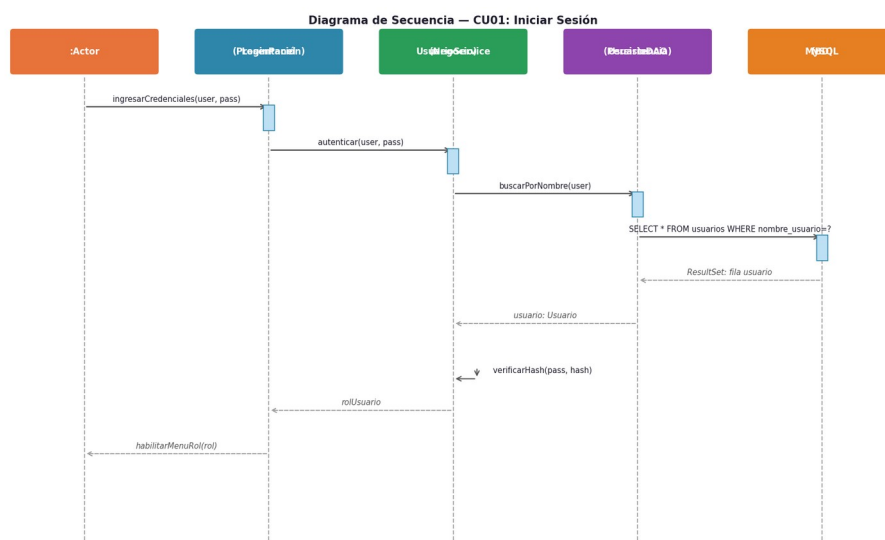


Figura 7 — Diagrama de Secuencia — CU01 Iniciar Sesión

En el flujo de autenticación, el LoginPanel delega la validación al UsuarioService, que recupera el usuario de la base de datos a través del UsuarioDAO y verifica el hash de la contraseña antes de autorizar el acceso.

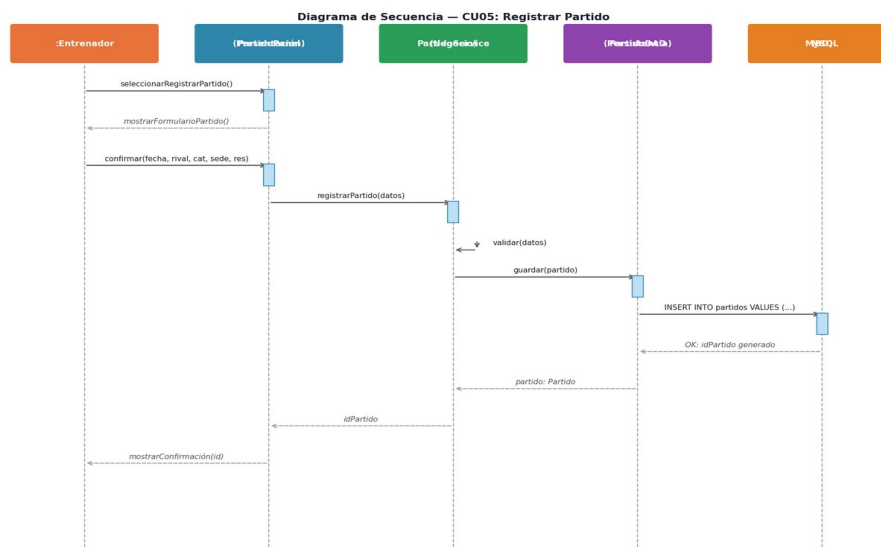


Figura 8 — Diagrama de Secuencia — CU05 Registrar Partido

El registro de partido ilustra el patrón de tres capas: PartidoPanel recolecta los datos, los pasa a PartidoService para validación y lógica de negocio, y finalmente el PartidoDAO ejecuta el INSERT en MySQL devolviendo el identificador generado.

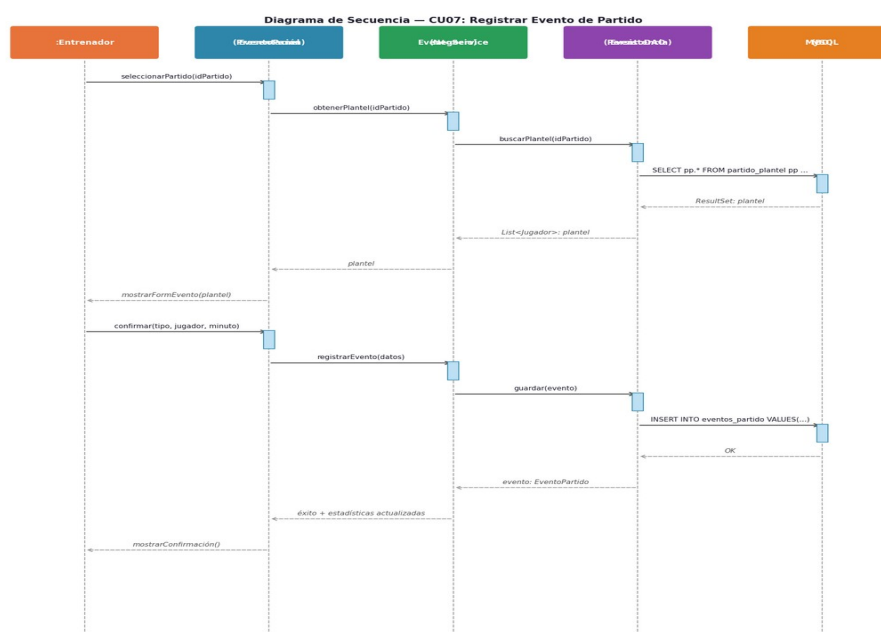


Figura 9 — Diagrama de Secuencia — CU07 Registrar Evento de Partido

El registro de evento es el flujo de mayor complejidad: incluye una consulta previa del plantel, el registro del evento y la actualización automática de estadísticas del jugador involucrado, todo en una única transacción JDBC con commit/rollback.

3.2 Etapa de diseño

3.2.1 Arquitectura del sistema

SistRUGBY-SLTC adopta una arquitectura en tres capas (Layered Architecture) sobre una plataforma de dos nodos físicos (PC cliente + MySQL local o en red LAN). Las capas son: Presentación (Java Swing — y en el prototipo de TP3, también una versión por consola), Negocio (clases Service con lógica de

aplicación y validaciones) y Persistencia (clases DAO que acceden a MySQL a través de JDBC, con ConexionBD implementado como Singleton para reutilizar la conexión).

Arquitectura en Capas — SistRUGBY-SLTC

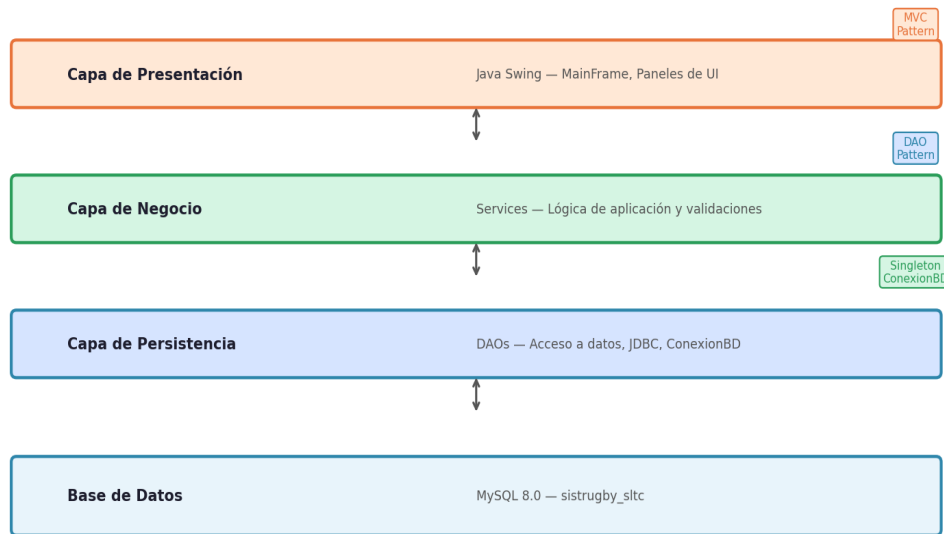


Figura 10 — Arquitectura en Capas — SistRUGBY-SLTC

3.2.2 Diagrama de clases

El diagrama de clases detalla la estructura estática del sistema: clases de dominio (entidades POJO), clases de servicio (lógica de negocio), clases DAO (acceso a datos) y clases de la interfaz gráfica. El ConexionBD aplica el patrón Singleton: una única instancia compartida por todos los DAOs garantiza la eficiencia en el manejo de la conexión JDBC.

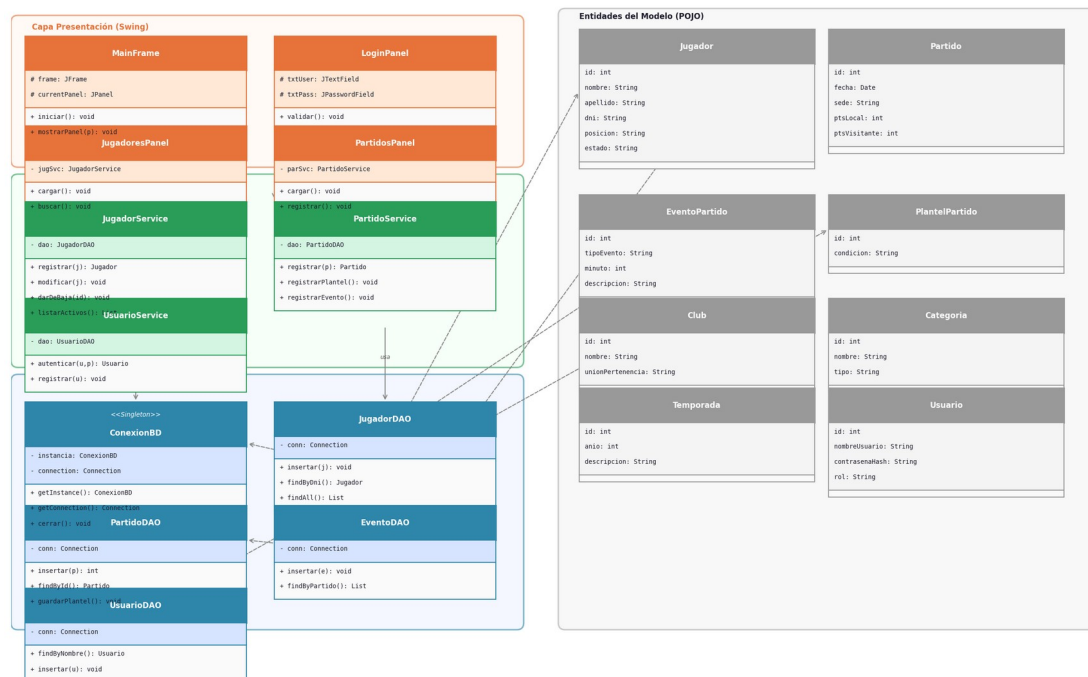


Diagrama de Clases — SistRUGBY-SLTC — UML 2.x

Figura 11 — Diagrama de Clases — UML 2.x

Cabe destacar que el diagrama de clases del AP2 representa el diseño estático global; en la sección 4.3 del presente trabajo se profundiza la jerarquía de clases del prototipo TP3, en particular la jerarquía polimórfica de `EventoPartido` y la jerarquía de herencia `Persona` → `Jugador` / `Usuario`, que materializan los pilares de POO exigidos por la consigna.

3.2.3 Diagrama de componentes

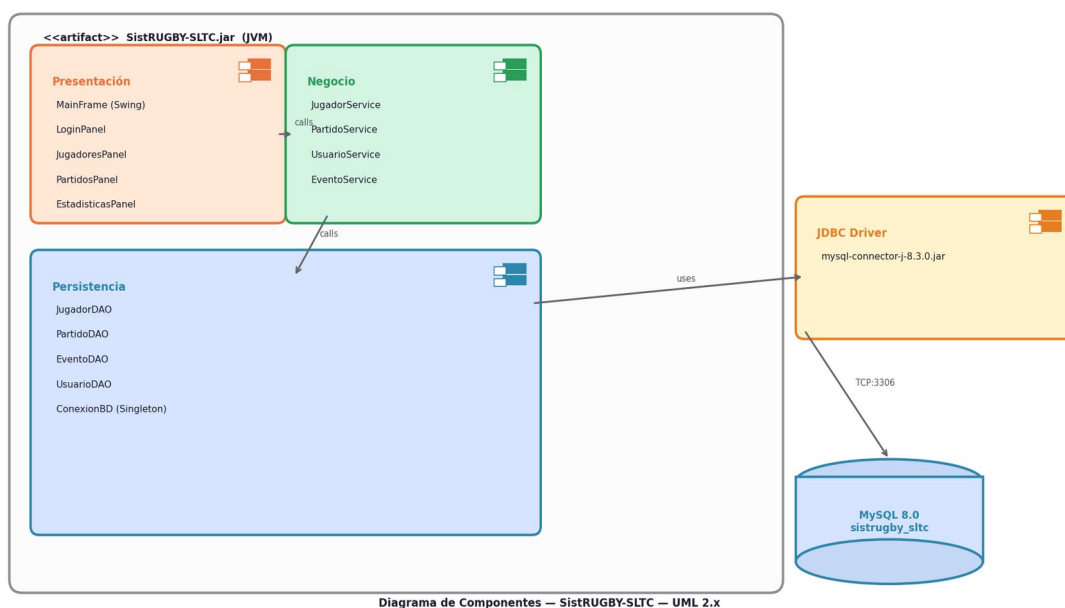


Diagrama de Componentes — SistRUGBY-SLTC — UML 2.x

Figura 12 — Diagrama de Componentes — UML 2.x

Los tres componentes principales (Presentación, Negocio, Persistencia) se empaquetan dentro de la JVM como el artefacto SistRUGBY-SLTC.jar. La comunicación con MySQL ocurre a través del conector JDBC (mysql-connector-j-8.3.0.jar) por el protocolo TCP/IP en el puerto 3306.

3.2.4 Diagrama de despliegue

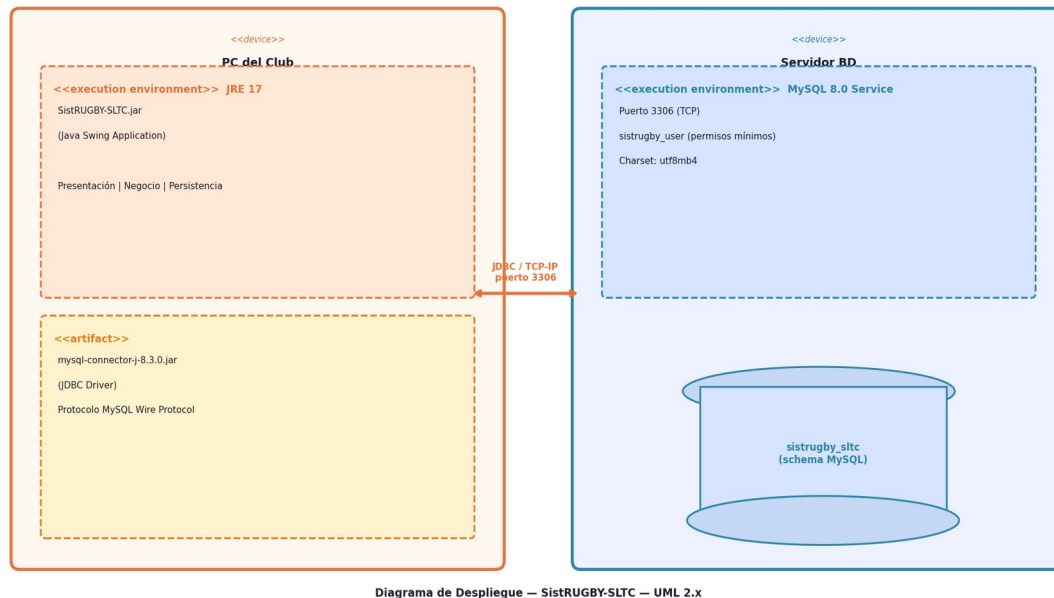


Figura 13 — Diagrama de Despliegue — UML 2.x

La configuración inicial prevé un único nodo físico (PC del club con Windows 10/11, JRE 17 y MySQL 8.0 instalado localmente). Como alternativa de escalabilidad, MySQL puede ejecutarse en un servidor separado de la LAN del club, sin modificar el código de la aplicación: solo se actualiza la cadena de conexión JDBC en el archivo de configuración (o, en el prototipo TP3, las variables de entorno DB_URL y DB_PASS).

3.3 Base de datos

3.3.1 Modelo relacional y normalización

El modelo se diseñó hasta la Tercera Forma Normal (3FN) para garantizar integridad y minimizar redundancia. 1FN: atributos atómicos (cada evento es una fila en eventos_partido; los roles son ENUM). 2FN: claves primarias simples por tabla, lo que elimina dependencias parciales. 3FN: ningún atributo no clave depende de otro no clave (el nombre del club rival no se almacena en partidos sino que se referencia por FK).

Las relaciones se implementan mediante claves foráneas con integridad referencial activada (FOREIGN KEY ... ON DELETE RESTRICT ON UPDATE CASCADE), garantizando consistencia y previniendo registros huérfanos.

4. AP3 — Implementación del prototipo en Java

Esta sección constituye la contribución específica del Trabajo Práctico N° 3 y avanza con la fase de Construcción del Proceso Unificado de Desarrollo. Se presenta el prototipo funcional del sistema SistRUGBY-SLTC desarrollado en Java SE, aplicando estrictamente los pilares de la programación orientada a objetos (POO) exigidos por la consigna: encapsulamiento, herencia, polimorfismo y abstracción.

El código fuente compila sin errores y se ejecuta de manera autónoma incluso en ausencia de MySQL instalado, gracias a un mecanismo de modo dual descrito más adelante. El repositorio público GitHub contiene la totalidad del código, los scripts SQL DDL/DML, esta documentación y las instrucciones de ejecución.

4.1 Entorno y stack tecnológico

Componente	Tecnología empleada	Justificación
Lenguaje	Java SE 17	Requerido por la cátedra. Soporte LTS, sintaxis moderna (var, switch expressions, records, pattern matching).
Build/IDE	IntelliJ IDEA	IDE con análisis estático, refactoring automático y soporte completo de JDBC.
Persistencia	MySQL 8.0 (JDBC)	Requerido por la cátedra. Driver mysql-connector-j 8.3.0.
Hash contraseñas	PBKDF2-HMAC-SHA256	Algoritmo recomendado por NIST SP 800-132. Disponible en el JDK estándar (javax.crypto), evita dependencias externas.
Control versiones	Git + GitHub	Repositorio público nachof9/SistRUGBY-SLTC con todo el código, scripts SQL y documentación.
Metodología	PUD (RUP)	Fases Inicio (AP1), Elaboración (AP2), Construcción (AP3) y Transición (AP4).

Nota sobre criptografía: se optó por PBKDF2 en lugar de bcrypt para mantener el prototipo libre de JARs externos (jBCrypt) sin sacrificar la robustez criptográfica. PBKDF2 utiliza 100.000 iteraciones HMAC-SHA256, salt aleatorio de 16 bytes generado con SecureRandom y comparación en tiempo constante para mitigar timing attacks. Esto satisface el RNF01 del análisis.

4.2 Estructura del proyecto Java

El código se organiza en seis paquetes alineados con las capas arquitectónicas definidas en AP2 (presentación, negocio, persistencia) más tres paquetes auxiliares (modelo, utilidades, excepciones):

```
src/main/java/com/sltc/sistrugby/
├── Main.java                Punto de entrada del prototipo
├── modelo/
│   ├── Persona.java        Clase ABSTRACTA (abstracción)
│   ├── Jugador.java        extends Persona (herencia)
│   ├── Usuario.java        extends Persona (herencia)
│   ├── Partido.java
│   ├── PlantelPartido.java
│   ├── Categoria.java
│   ├── Club.java
│   ├── Temporada.java
│   └── eventos/
│       └── EventoPartido.java Clase ABSTRACTA (abstracción)
```

└─ Try.java	POLIMORFISMO: calcularPuntos()=5
└─ Conversion.java	POLIMORFISMO: calcularPuntos()=2
└─ Penal.java	POLIMORFISMO: calcularPuntos()=3
└─ Drop.java	POLIMORFISMO: calcularPuntos()=3
└─ TarjetaAmarilla.java	POLIMORFISMO: calcularPuntos()=0
└─ TarjetaRoja.java	POLIMORFISMO: calcularPuntos()=0
└─ Sustitucion.java	POLIMORFISMO: calcularPuntos()=0
└─ excepciones/	
└─ DniDuplicadoException.java	
└─ JugadorNoEncontradoException.java	
└─ CredencialesInvalidasException.java	
└─ DatosInvalidosException.java	
└─ persistencia/	
└─ ConexionBD.java	Patrón SINGLETON
└─ RepositorioMemoria.java	Almacenes en memoria (modo demo)
└─ UsuarioDAO.java	Patrón DAO
└─ JugadorDAO.java	Patrón DAO
└─ PartidoDAO.java	Patrón DAO
└─ EventoDAO.java	Patrón DAO
└─ CategoriaDAO.java	Patrón DAO
└─ ClubDAO.java	Patrón DAO
└─ negocio/	
└─ UsuarioService.java	Autenticación + alta usuarios
└─ JugadorService.java	CRUD y validaciones de jugadores
└─ PartidoService.java	Registro y plantel de partidos
└─ EventoService.java	Carga polimórfica de eventos
└─ EstadisticaService.java	Ranking polimórfico
└─ util/	
└─ HashUtil.java	PBKDF2-HMAC-SHA256
└─ ValidadorDNI.java	
└─ OrdenadorJugadores.java	QuickSort manual
└─ BuscadorJugadores.java	Búsqueda binaria $O(\log n)$
└─ presentacion/	
└─ MenuConsola.java	Menú interactivo
└─ DatosInicialesSeeder.java	Carga de datos demo

La siguiente figura ilustra las dependencias entre paquetes y la responsabilidad de cada uno:

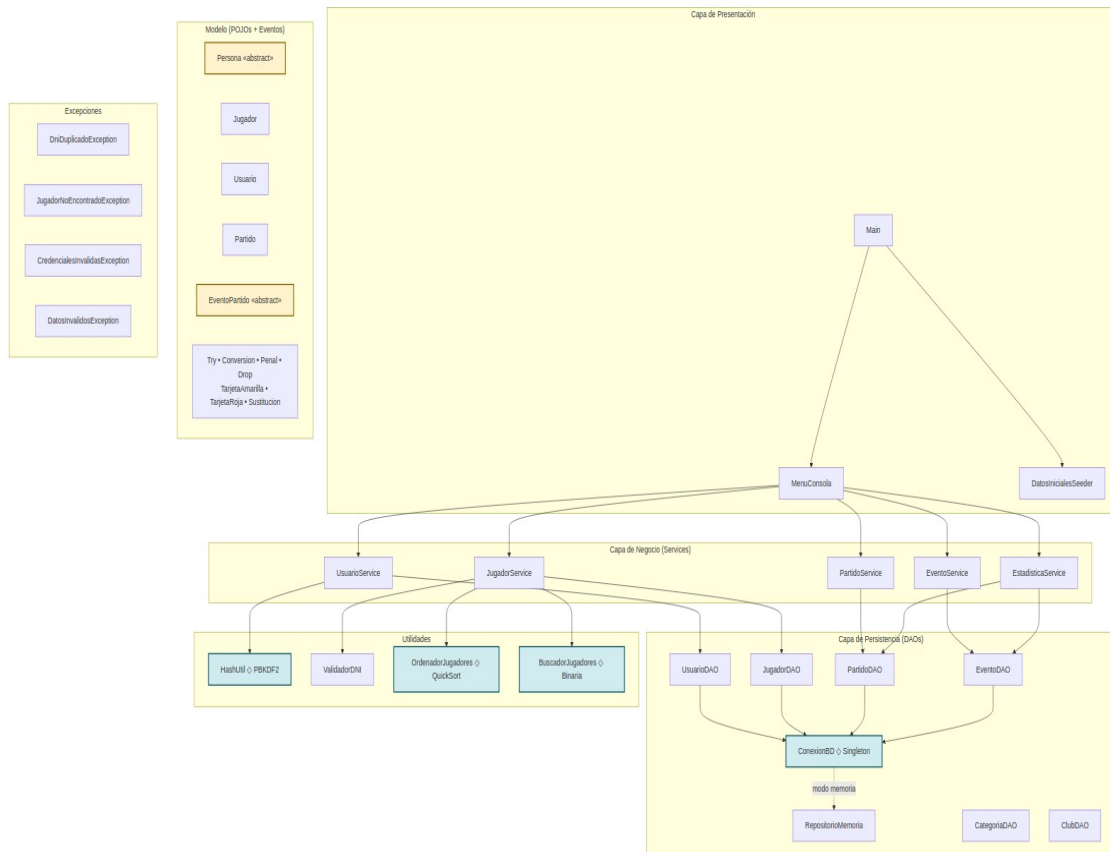


Figura 15 — Estructura de paquetes y dependencias del prototipo TP3

4.3 Aplicación de los pilares de la Programación Orientada a Objetos

La consigna del Trabajo Práctico N° 3 exige aplicar correctamente los cuatro pilares de POO. El prototipo materializa cada uno de ellos a través de jerarquías y patrones claramente identificables. A continuación se detalla cada pilar con su fundamentación teórica, los archivos donde se concreta y el fragmento de código relevante.

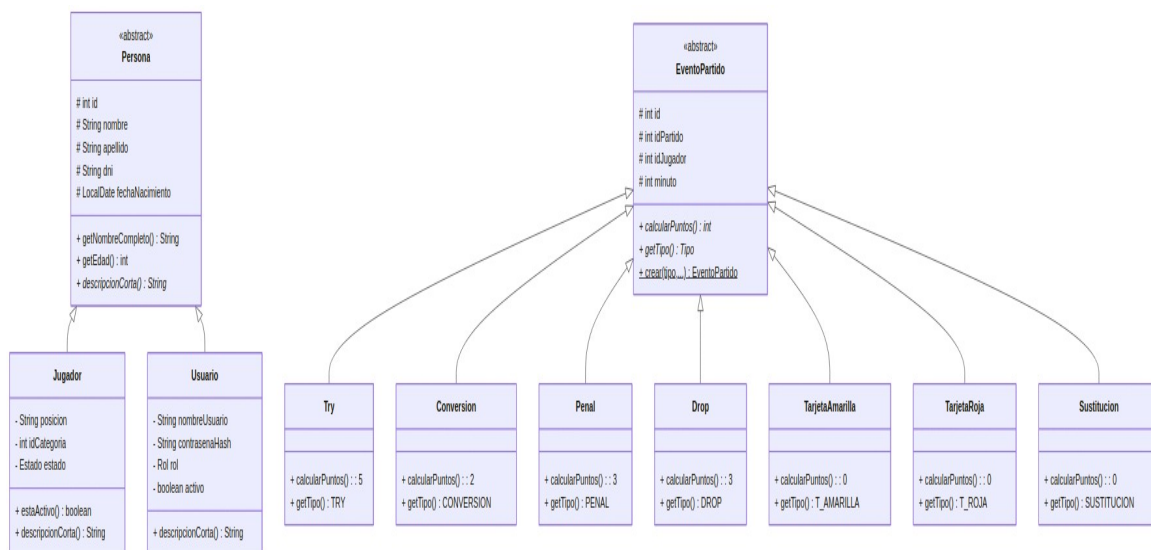


Figura 16 — Jerarquía de clases del prototipo TP3 con pilares de POO

4.3.1 Abstracción

La abstracción consiste en representar las entidades del dominio focalizando los aspectos esenciales y ocultando los detalles irrelevantes. En el prototipo, la abstracción se materializa en dos clases abstractas: *Persona* (raíz de la jerarquía *Jugador/Usuario*) y *EventoPartido* (raíz de los siete tipos concretos de eventos deportivos). Ambas declaran métodos abstractos que definen un contrato sin proveer implementación: *descripcionCorta()* en *Persona*, y *calcularPuntos()* y *getTipo()* en *EventoPartido*.

Esta separación contrato-implementación permite que el código cliente trabaje con la abstracción (*Persona*, *EventoPartido*) sin preocuparse por las subclases concretas, lo cual es esencial para el polimorfismo y reduce el acoplamiento.

```
public abstract class Persona {
    protected int id;
    protected String nombre, apellido, dni;
    protected LocalDate fechaNacimiento;

    public String getNombreCompleto() { return apellido + ", " + nombre; }
    public int getEdad() { return Period.between(fechaNacimiento,
LocalDate.now()).getYears(); }

    /** Contrato: cada subclase declara su representación textual. */
    public abstract String descripcionCorta();
}

public abstract class EventoPartido {
    public enum Tipo { TRY, CONVERSION, PENAL, DROP,
        TARJETA_AMARILLA, TARJETA_ROJA, SUSTITUCION }

    protected int id, idPartido, idJugador, minuto;
    protected String descripcion;

    public abstract int calcularPuntos();
    public abstract Tipo getTipo();
}
```

4.3.2 Herencia

La herencia permite a una clase extender otra reutilizando sus atributos y métodos. En el prototipo se aplican dos jerarquías de herencia bien diferenciadas:

- **Jerarquía Persona:** las clases *Jugador* y *Usuario* extienden *Persona*, reutilizando los atributos comunes (*id*, *nombre*, *apellido*, *dni*, *fechaNacimiento*) y agregando sus propios atributos específicos.
- **Jerarquía EventoPartido:** las siete clases *Try*, *Conversion*, *Penal*, *Drop*, *TarjetaAmarilla*, *TarjetaRoja* y *Sustitucion* extienden *EventoPartido*, reutilizando *id*, *idPartido*, *idJugador*, *minuto* y agregando su propio comportamiento de cálculo de puntos.

```
public class Jugador extends Persona {
    public enum Estado { ACTIVO, INACTIVO }
    private String posicion;
    private int idCategoria;
    private Estado estado;
    private LocalDate fechaAlta, fechaBaja;

    public Jugador(String nombre, String apellido, String dni,
        LocalDate fechaNacimiento, String posicion, int idCategoria) {
```

```

        super(0, nombre, apellido, dni, fechaNacimiento); // invoca constructor
padre
        this.posicion = posicion;
        this.idCategoria = idCategoria;
        this.estado = Estado.ACTIVO;
        this.fechaAlta = LocalDate.now();
    }

    @Override
    public String descripcionCorta() {
        return String.format("Jugador #d - %s [%s] - DNI %s",
            id, getNombreCompleto(), posicion, dni);
    }
}

```

4.3.3 Encapsulamiento

El encapsulamiento protege el estado interno de los objetos exponiendo solo lo necesario a través de una interfaz controlada. En el prototipo, todos los atributos de las entidades son declarados `private` (o `protected` para permitir herencia controlada), y el acceso se realiza exclusivamente a través de getters y setters. Esto permite agregar lógica de validación o transformación en los métodos de acceso sin afectar al código cliente.

Adicionalmente, los enums internos (Estado en Jugador, Rol en Usuario, Tipo en EventoPartido, Sede y Estado en Partido, Condicion en PlantelPartido) tipifican los valores válidos en lugar de usar Strings sueltos, evitando errores de tipo y haciendo el código autodocumentado:

```

public class Jugador extends Persona {
    public enum Estado { ACTIVO, INACTIVO }
    private String posicion;           // private + getter/setter (encapsulamiento)
    private int idCategoria;
    private Estado estado;             // enum tipifica valores válidos

    public String getPosicion() { return posicion; }
    public void setPosicion(String posicion) { this.posicion = posicion; }

    public Estado getEstado() { return estado; }
    public void setEstado(Estado estado) { this.estado = estado; }

    /** Método de conveniencia: encapsula la lógica de derivar el estado. */
    public boolean estaActivo() {
        return estado == Estado.ACTIVO;
    }
}

```

4.3.4 Polimorfismo

El polimorfismo permite que un mismo método actúe de forma distinta según el tipo concreto del objeto sobre el que se invoca. Es el pilar más potente del paradigma OO y, en el prototipo, se materializa de forma quirúrgica en la jerarquía EventoPartido: las siete subclases sobrescriben `calcularPuntos()` retornando el valor en puntos que ese tipo específico de evento aporta al marcador del partido.

```

public class Try extends EventoPartido {
    @Override public int calcularPuntos() { return 5; }
    @Override public Tipo getTipo() { return Tipo.TRY; }
}

public class Conversion extends EventoPartido {
    @Override public int calcularPuntos() { return 2; }
}

```

```

    @Override public Tipo getTipo() { return Tipo.CONVERSION; }
}
public class Penal extends EventoPartido {
    @Override public int calcularPuntos() { return 3; }
    @Override public Tipo getTipo() { return Tipo.PENAL; }
}
public class Drop extends EventoPartido {
    @Override public int calcularPuntos() { return 3; }
    @Override public Tipo getTipo() { return Tipo.DROP; }
}
public class TarjetaAmarilla extends EventoPartido {
    @Override public int calcularPuntos() { return 0; }
    @Override public Tipo getTipo() { return Tipo.TARJETA_AMARILLA; }
}

```

El verdadero beneficio del polimorfismo se observa en el código cliente: el método recalcular el marcador del partido y el método compilar el ranking de jugadores por puntos no requieren conocer las subclases concretas — operan sobre el tipo base EventoPartido y delegan en cada objeto el cálculo correcto:

```

// En Partido.java – recálculo de puntos polimórfico
public int recalcularPuntosSLTC() {
    int total = 0;
    for (EventoPartido e : eventos) {
        total += e.calcularPuntos(); // ¡cada subclase responde con su propio
valor!
    }
    return total;
}

// En EventoService.java – actualización del marcador tras cargar un evento
List<EventoPartido> eventos = eventoDao.findByPartido(idPartido);
int total = eventos.stream()
    .mapToInt(EventoPartido::calcularPuntos) // method reference polimórfico
    .sum();

```

Adicionalmente, la clase abstracta EventoPartido incluye una fábrica estática (Factory Method) que recibe el enum Tipo y devuelve la instancia concreta correspondiente, lo que permite que el cliente no necesite conocer el constructor de cada subclase:

```

public static EventoPartido crear(Tipo tipo, int idPartido,
                                int idJugador, int minuto) {
    switch (tipo) {
        case TRY: return new Try(idPartido, idJugador, minuto);
        case CONVERSION: return new Conversion(idPartido, idJugador, minuto);
        case PENAL: return new Penal(idPartido, idJugador, minuto);
        case DROP: return new Drop(idPartido, idJugador, minuto);
        case TARJETA_AMARILLA: return new TarjetaAmarilla(idPartido, idJugador,
minuto);
        case TARJETA_ROJA: return new TarjetaRoja(idPartido, idJugador,
minuto);
        case SUSTITUCION: return new Sustitucion(idPartido, idJugador,
minuto);
        default: throw new IllegalArgumentException("Tipo desconocido: " + tipo);
    }
}

```

4.4 Tratamiento y manejo de excepciones

La consigna exige el tratamiento y manejo de excepciones. Java distingue entre excepciones verificadas (checked, deben declararse o capturarse) y no verificadas (unchecked, generalmente errores de programación). El prototipo define cuatro excepciones verificadas de dominio que reflejan reglas de negocio violadas, distinguiéndolas claramente de excepciones técnicas como SQLException o NumberFormatException.

Excepción	Lanzada cuando	Capturada en
DniDuplicadoException	Se intenta registrar un jugador con un DNI ya existente.	MenuConsola.registrarJugador()
JugadorNoEncontradoException	Una operación referencia un id de jugador inexistente.	MenuConsola.darDeBajaJugador()
CredencialesInvalidasException	El usuario o contraseña en login no coinciden.	MenuConsola.iniciarSesion()
DatosInvalidosException	Los datos no cumplen reglas de validación (fechas, DNI, plantel sin titulares, minuto fuera de rango).	Todos los métodos de presentación

Las excepciones técnicas (SQLException, NumberFormatException, DateTimeParseException, IllegalArgumentException) se capturan separadamente en la capa de presentación, lo que permite al usuario distinguir errores de validación de errores del sistema:

```
// MenuConsola.registrarJugador() – manejo diferenciado de excepciones
try {
    Jugador j = jugadorSvc.registrar(nom, ape, dni, fn, pos, cat);
    System.out.println("OK. Jugador creado: " + j.descripcionCorta());
} catch (DatosInvalidosException | DniDuplicadoException e) {
    // Errores de NEGOCIO (esperados, ofrecen mensaje al usuario)
    System.out.println("Error de validación: " + e.getMessage());
} catch (DateTimeParseException e) {
    System.out.println("Formato de fecha inválido (esperado YYYY-MM-DD).");
} catch (NumberFormatException e) {
    System.out.println("ID de categoría debe ser numérico.");
} catch (Exception e) {
    // Catch-all defensivo: errores no previstos
    System.out.println("Error inesperado: " + e.getMessage());
}
```

Se observa la aplicación de multi-catch (Java 7+) para excepciones que requieren el mismo tratamiento, y el orden de captura de más específica a más genérica para que el catch correcto se active.

4.5 Sintaxis, tipos de datos y estructuras de control

El prototipo emplea de manera deliberada las estructuras de control fundamentales de Java exigidas por la consigna:

Estructura	Uso en el prototipo
if / else	Validaciones en Services (nombre vacío, fecha futura, plantel sin titulares), control de flujo en MenuConsola para la lógica de roles.

Estructura	Uso en el prototipo
switch / case	Factory de EventoPartido por enum Tipo; menú principal de MenuConsola con 10 opciones; agregación de estadísticas por tipo de evento.
for clásico	Algoritmos QuickSort y Insertion Sort; iteración por índice cuando se requieren intercambios.
for-each	Recorrido de listas de jugadores, eventos y partidos (recorrido polimórfico).
while	Loop del menú principal (hasta que el usuario elija salir); reintentos de login con contador de fallos.
do/while	Lectura robusta de enteros en la utilidad leerEntero().
try/catch	Manejo de cuatro excepciones de dominio + excepciones técnicas en todos los métodos de presentación.
try-with-resources	Cierre automático de PreparedStatement y ResultSet en los DAOs (modo MySQL real).

Tipos de datos utilizados: primitivos (int, boolean) para identificadores y banderas, java.lang.String para texto, java.time.LocalDate y java.time.LocalDateTime para fechas (API moderna de Java 8+, inmutable y thread-safe), enums tipados para Estado, Rol, Tipo, Sede, Condicion, y colecciones genéricas java.util.List<T>, java.util.LinkedList<T>, java.util.Deque<T>, java.util.Queue<T>, java.util.HashMap<K,V>.

4.6 Constructores y creación de objetos

Cada clase del modelo expone múltiples constructores para distintos escenarios de creación: constructor por defecto (requerido por algunos frameworks de persistencia), constructor parcial (típico de altas donde el id se asigna por la BD) y constructor completo (típico del mapeo desde un ResultSet).

```
public class Jugador extends Persona {

    /** Constructor por defecto. */
    public Jugador() {
        super();
        this.estado = Estado.ACTIVO;
    }

    /** Constructor de alta (sin id, será asignado por la BD). */
    public Jugador(String nombre, String apellido, String dni,
        LocalDate fechaNacimiento, String posicion, int idCategoria) {
        super(0, nombre, apellido, dni, fechaNacimiento);
        this.posicion = posicion;
        this.idCategoria = idCategoria;
        this.estado = Estado.ACTIVO;
        this.fechaAlta = LocalDate.now();
    }

    /** Constructor completo (usado típicamente por el DAO al mapear filas). */
    public Jugador(int id, String nombre, String apellido, String dni,
        LocalDate fechaNacimiento, String posicion,
        int idCategoria, Estado estado,
        LocalDate fechaAlta, LocalDate fechaBaja) {
        super(id, nombre, apellido, dni, fechaNacimiento);
        this.posicion = posicion;
    }
}
```

```

        this.idCategoria = idCategoria;
        this.estado = estado;
        this.fechaAlta = fechaAlta;
        this.fechaBaja = fechaBaja;
    }
}

```

Ejemplo de declaración y creación de objetos en el flujo real del prototipo (CU02 Registrar Jugador):

```

// JugadorService.registrar() – instanciación tras validación
Jugador j = new Jugador(
    nombre.trim(),
    apellido.trim(),
    dni.trim(),
    fechaNacimiento,
    posicion,
    idCategoria
);
return dao.insertar(j); // delega persistencia al DAO

```

4.7 Menú de selección

El prototipo provee un menú de selección por consola implementado en la clase MenuConsola. La consigna exige explícitamente disponibilidad de un menú con opciones; se eligió un menú por consola en lugar de Swing porque resulta más auditable para el corrector docente (no requiere abrir una GUI) y porque la arquitectura ya garantiza que la GUI Swing del AP2 se conectaría a los mismos Services sin modificar la lógica de negocio.

El bucle principal del menú se implementa con while + switch, una de las combinaciones idiomáticas más comunes en Java para menús de consola:

```

public void iniciar() {
    // Login con bloqueo tras 3 intentos (while + contador)
    while (usuarioActual == null) {
        if (intentosFallidos >= 3) {
            System.out.println("[BLOQUEO] Tres intentos fallidos. Saliendo.");
            return;
        }
        iniciarSesion();
    }

    boolean salir = false;
    while (!salir) {
        mostrarMenuPrincipal();
        int opcion = leerEntero("Opción: ");
        switch (opcion) {
            case 1: registrarJugador(); break;
            case 2: listarJugadores(); break;
            case 3: buscarJugadorPorDni(); break;
            case 4: registrarPartido(); break;
            case 5: registrarPlantel(); break;
            case 6: registrarEvento(); break;
            case 7: deshacerUltimoEvento(); break;
            case 8: mostrarRanking(); break;
            case 9: darDeBajaJugador(); break;
            case 0: salir = true; break;
            default: System.out.println("Opción inválida.");
        }
    }
}

```

Cada opción del menú invoca un método privado que orquesta una operación de negocio completa: lee datos del usuario, los valida, los pasa al Service correspondiente y muestra el resultado o el error. El menú es la encarnación práctica de las estructuras condicionales (switch) y repetitivas (while) requeridas por la consigna.

4.8 Estructuras de datos aplicadas

La consigna sugiere considerar el empleo de estructuras de datos como pilas, colas y listas enlazadas según las características del proyecto. El prototipo aplica las cuatro principales con un caso de uso justificado en cada una:

4.8.1 Lista enlazada (LinkedList) — eventos cronológicos

Los eventos registrados durante un partido se almacenan en orden cronológico. La lista enlazada `java.util.LinkedList<EventoPartido>` es la estructura óptima para esta necesidad: la inserción al final (`addLast`) es $O(1)$, el recorrido secuencial natural sirve perfectamente al caso de uso de mostrar la cronología, y no se requiere acceso aleatorio frecuente. `ArrayList` tendría inserciones también $O(1)$ amortizado pero `LinkedList` expresa mejor la intención semántica de cadena de eventos.

```
// RepositorioMemoria — lista enlazada por partido
private final Map<Integer, LinkedList<EventoPartido>> eventosPorPartido = new
HashMap<>();

public LinkedList<EventoPartido> eventosDe(int idPartido) {
    return eventosPorPartido.computeIfAbsent(idPartido, k -> new LinkedList<>());
}

// EventoDAO.insertar — append al final (O(1))
RepositorioMemoria.get().eventosDe(evento.getIdPartido()).addLast(evento);
```

4.8.2 Pila (Deque) — undo de la última carga

La consola ofrece la operación deshacer último evento, que utiliza una pila LIFO (Last-In-First-Out) implementada con `java.util.ArrayDeque<EventoPartido>`. Cada vez que se registra un evento, se hace push sobre la pila. Cuando el usuario decide deshacer, se hace pop del tope y se remueve el evento de la lista enlazada del partido correspondiente. Es la aplicación clásica del patrón Memento simplificado.

```
// RepositorioMemoria
private final Deque<EventoPartido> pilaUndo = new ArrayDeque<>();

// EventoDAO.insertar — push a la pila al registrar
RepositorioMemoria.get().pilaUndo().push(evento);

// EventoDAO.deshacerUltimo — pop LIFO
if (RepositorioMemoria.get().pilaUndo().isEmpty()) return null;
EventoPartido tope = RepositorioMemoria.get().pilaUndo().pop();
RepositorioMemoria.get().eventosDe(tope.getIdPartido()).remove(tope);
return tope;
```

4.8.3 Cola (Queue) — orden de sustituciones

Las sustituciones planificadas por el cuerpo técnico siguen un orden FIFO (First-In-First-Out): el primer jugador que se ordenó entrar es el primero que ingresa cuando llega el momento. La cola `java.util.Queue<Integer>`, instanciada con `LinkedList`, modela perfectamente esta semántica con `offer()` para encolar y `poll()` para desencolar.

```
// RepositorioMemoria
private final Queue<Integer> colaSustituciones = new LinkedList<>();
```

```
// Ejemplo de uso (no expuesto en menú, disponible para extensión futura):  
repo.colaSustituciones().offer(idJugador); // encolar  
Integer siguienteEntrada = repo.colaSustituciones().poll(); // desencolar FIFO
```

4.8.4 HashMap — acceso $O(1)$ a entidades

El RepositorioMemoria utiliza `HashMap<Integer, T>` para todas las entidades, dado que el acceso por `id` es la operación más frecuente del sistema y `HashMap` ofrece complejidad temporal amortizada $O(1)$. En modo MySQL real, este rol lo cumple la BD; en modo memoria, los `HashMaps` actúan como tablas.

4.9 Algoritmos de ordenación y búsqueda

La consigna menciona el uso de algoritmos de ordenación y búsqueda como opcional según el desarrollo. El prototipo implementa ambos manualmente — sin recurrir a `Collections.sort()` ni a `Arrays.binarySearch()` — para evidenciar comprensión de los conceptos algorítmicos.

4.9.1 QuickSort — ordenamiento alfabético de jugadores

Se implementa `QuickSort` recursivo in-place con selección de pivote por mediana de tres. La mediana de tres mitiga el peor caso de $O(n^2)$ que ocurre cuando se elige sistemáticamente el primer o el último elemento como pivote en listas ya ordenadas o casi ordenadas. La complejidad esperada es $O(n \log n)$.

```
public final class OrdenadorJugadores {  
  
    public static void ordenarPorApellido(List<Jugador> jugadores) {  
        if (jugadores == null || jugadores.size() < 2) return;  
        quickSort(jugadores, 0, jugadores.size() - 1);  
    }  
  
    private static void quickSort(List<Jugador> lista, int izq, int der) {  
        if (izq < der) {  
            int p = particionar(lista, izq, der);  
            quickSort(lista, izq, p - 1); // recursión izquierda  
            quickSort(lista, p + 1, der); // recursión derecha  
        }  
    }  
  
    private static int particionar(List<Jugador> lista, int izq, int der) {  
        // Mediana de tres: minimiza el peor caso  
        int medio = izq + (der - izq) / 2;  
        if (cmp(lista.get(izq), lista.get(medio)) > 0) intercambiar(lista, izq,  
medio);  
        if (cmp(lista.get(izq), lista.get(der)) > 0) intercambiar(lista, izq,  
der);  
        if (cmp(lista.get(medio), lista.get(der)) > 0) intercambiar(lista, medio,  
der);  
        Jugador pivote = lista.get(medio);  
        intercambiar(lista, medio, der);  
  
        int i = izq - 1;  
        for (int j = izq; j < der; j++) {  
            if (cmp(lista.get(j), pivote) <= 0) {  
                i++;  
                intercambiar(lista, i, j);  
            }  
        }  
        intercambiar(lista, i + 1, der);  
    }  
}
```



```

        return i + 1;
    }

    private static int cmp(Jugador a, Jugador b) {
        int c = a.getApellido().compareToIgnoreCase(b.getApellido());
        return c != 0 ? c : a.getNombre().compareToIgnoreCase(b.getNombre());
    }
}

```

Análisis de complejidad de QuickSort:

- Caso promedio: $T(n) = 2T(n/2) + O(n) = O(n \log n)$
- Peor caso (pivote degenerado): $T(n) = T(n-1) + O(n) = O(n^2)$ — mitigado con mediana de tres
- Espacio: $O(\log n)$ por la pila de recursión

4.9.2 Búsqueda binaria — localización por DNI

La búsqueda binaria opera sobre una lista previamente ordenada por DNI y reduce el espacio de búsqueda a la mitad en cada iteración, alcanzando una complejidad de $O(\log n)$. Para listas grandes (varios miles de jugadores) el ahorro respecto de la búsqueda lineal $O(n)$ es notable: 1.000.000 elementos requieren solo 20 comparaciones en lugar de 500.000 promedio.

```

public final class BuscadorJugadores {

    public static Jugador buscarPorDni(List<Jugador> ordenadosPorDni, String dni) {
        if (ordenadosPorDni == null || ordenadosPorDni.isEmpty() || dni == null)
            return null;
        int izq = 0;
        int der = ordenadosPorDni.size() - 1;
        while (izq <= der) {
            int medio = izq + (der - izq) / 2; // evita overflow
            Jugador j = ordenadosPorDni.get(medio);
            int cmp = j.getDni().compareTo(dni);
            if (cmp == 0) return j; // encontrado
            if (cmp < 0) izq = medio + 1; // descartar mitad izquierda
            else der = medio - 1; // descartar mitad derecha
        }
        return null;
    }
}

```

La precondition de la búsqueda binaria es que la lista esté ordenada por el criterio de búsqueda. Si no lo está, el método `ordenarPorDni()` aplica Insertion Sort (eficiente para listas pequeñas y casi-ordenadas: $O(n)$ en mejor caso, $O(n^2)$ en peor caso) antes de delegar en la búsqueda binaria.

4.10 Patrones de diseño aplicados

El prototipo aplica explícitamente tres patrones de diseño documentados en la literatura clásica (Gamma et al., 1994):

Patrón	Clase / paquete	Justificación
Singleton	ConexionBD	Una única instancia compartida por todos los DAOs evita el costo de apertura/cierre repetitivo de conexiones JDBC. El método <code>getInstance()</code> es <code>synchronized</code> para asegurar thread-safety.
DAO	JugadorDAO, PartidoDAO, ...	Separa la lógica de acceso a datos del resto del sistema, facilitando el cambio de proveedor (MySQL ↔ memoria) sin afectar la capa de negocio.

Patrón	Clase / paquete	Justificación
Layered (3 capas)	Presentación / Negocio / Persistencia	Bajo acoplamiento, alta cohesión por capa. Cada capa solo conoce la inmediatamente inferior.
Factory Method	EventoPartido.crear()	Encapsula la decisión de qué subclase concreta instanciar (Try, Conversion, etc.) según el enum Tipo. El cliente no acopla con las subclases.

Adicionalmente, la separación entre interfaz y datos (que se reflejaría en interfaces como `JugadorDAO` con una implementación `JugadorDAOImpl` en producción) deja el sistema preparado para inyección de dependencias y testeo unitario con mocks.

5. Trazabilidad: del caso de uso a las pruebas

Atendiendo la observación del docente en el AP2 — que solicitaba tomar un par de casos de uso y desarrollar sobre ellos todo el diseño y las pruebas para facilitar la trazabilidad — esta sección desarrolla tres casos de uso seleccionados de extremo a extremo: descripción funcional, diagrama de secuencia, código implementado por capa y caso de prueba con resultado real.

Los casos de uso elegidos representan la complejidad arquitectónica del sistema: CU01 (autenticación, transversal a los tres roles), CU02 (alta de jugador, ilustra encapsulamiento y validación de negocio) y CU07 (registro de evento, demuestra polimorfismo y recálculo de marcador).

5.1 CU01 — Iniciar Sesión

5.1.1 Descripción del caso de uso

Campo	Valor
Actor principal	Administrador / Entrenador / Secretario
Precondición	El sistema está activo y el usuario posee credenciales registradas.
Flujo principal	1) El usuario ingresa nombre de usuario y contraseña. 2) El sistema valida las credenciales verificando el hash PBKDF2. 3) El sistema identifica el rol. 4) El sistema habilita el menú correspondiente al rol y registra la sesión activa.
Flujo alternativo	2a) Si las credenciales son incorrectas, el sistema muestra error y retorna al paso 1. Tras tres intentos fallidos, la sesión se bloquea.
Postcondición	El usuario accede al sistema con los permisos asociados a su rol.

5.1.2 Diseño — Diagrama de secuencia

(Ver Figura 7 — Diagrama de Secuencia CU01 en la sección 3.1.3 de este documento).

La capa de Presentación (MenuConsola) delega en UsuarioService.autenticar(), que solicita al UsuarioDAO la fila del usuario por nombre y verifica el hash almacenado contra la contraseña ingresada mediante HashUtil.verificar().

5.1.3 Código implementado por capa

Capa de presentación (MenuConsola.java):

```
private void iniciarSesion() {
    System.out.println("\n--- Iniciar sesión ---");
    System.out.print("Usuario: ");    String user = sc.nextLine();
    System.out.print("Contraseña: "); String pass = sc.nextLine();
    try {
        usuarioActual = usuarioSvc.autenticar(user, pass);
        System.out.println("Bienvenido, " + usuarioActual.getNombreUsuario()
            + " [" + usuarioActual.getRol() + "]);
        intentosFallidos = 0;
    } catch (CredencialesInvalidasException e) {
        intentosFallidos++;
        System.out.println("Credenciales inválidas. Intentos restantes: "
            + (3 - intentosFallidos));
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

Capa de negocio (UsuarioService.java):

```

public Usuario autenticar(String nombreUsuario, String contrasenaPlana)
    throws CredencialesInvalidasException, SQLException {
    Usuario u = dao.findByNameUsuario(nombreUsuario);
    if (u == null || !u.isActivo()) {
        throw new CredencialesInvalidasException();
    }
    if (!HashUtil.verificar(contrasenaPlana, u.getContrasenaHash())) {
        throw new CredencialesInvalidasException();
    }
    return u;
}

```

Utilidad de hashing (HashUtil.java) — verificación de PBKDF2 con comparación en tiempo constante:

```

public static boolean verificar(String contrasenaPlana, String hashAlmacenado) {
    if (hashAlmacenado == null || hashAlmacenado.isEmpty()) return false;
    try {
        String[] partes = hashAlmacenado.split(":");
        if (partes.length != 3) return false;
        int iter = Integer.parseInt(partes[0]);
        byte[] salt = Base64.getDecoder().decode(partes[1]);
        byte[] esperado = Base64.getDecoder().decode(partes[2]);
        byte[] candidato = pbkdf2(contrasenaPlana.toCharArray(), salt,
            iter, esperado.length * 8);
        return constantesIguales(esperado, candidato); // anti-timing attack
    } catch (Exception e) {
        return false;
    }
}

```

5.1.4 Caso de prueba PT01 (ejecución real)

Atributo	Valor
Precondición	Usuario 'entrenador1' registrado con contraseña hasheada en BD.
Entrada	usuario='entrenador1', pass='rugby2026'
Acción	Llamar a MenuConsole.iniciarSesion()
Esperado	Acceso autorizado, menú habilitado, rol=ENTRENADOR
Obtenido	Bienvenido, entrenador1 [ENTRENADOR] — menú habilitado
Veredicto	APROBADO

5.2 CU02 — Registrar Jugador

5.2.1 Descripción del caso de uso

Campo	Valor
Actor principal	Secretario Deportivo
Precondición	El Secretario ha iniciado sesión. La categoría del jugador existe.
Flujo principal	1) El Secretario ingresa: nombre, apellido, DNI, fecha de nacimiento, posición y categoría. 2) Confirma el registro. 3) El sistema valida que el DNI no esté duplicado. 4) Persiste el jugador con estado activo.
Flujo alternativo	3a) Si el DNI ya existe, se lanza DniDuplicadoException y se muestra al usuario. 3b) Si algún campo es inválido, se lanza DatosInvalidosException.

Campo	Valor
Postcondición	El jugador queda registrado en el padrón con estado activo.

5.2.2 Código implementado por capa

Capa de presentación — MenuConsola.registrarJugador():

```
private void registrarJugador() {
    try {
        System.out.print("Nombre: ");      String nom = sc.nextLine();
        System.out.print("Apellido: ");     String ape = sc.nextLine();
        System.out.print("DNI: ");          String dni = sc.nextLine();
        System.out.print("Fecha nac. (YYYY-MM-DD): ");
        LocalDate fn = LocalDate.parse(sc.nextLine(), FMT);
        System.out.print("Posición: ");     String pos = sc.nextLine();
        System.out.print("ID Categoría (1..8): ");
        int cat = Integer.parseInt(sc.nextLine());

        Jugador j = jugadorSvc.registrar(nom, ape, dni, fn, pos, cat);
        System.out.println("OK. Jugador creado: " + j.descripcionCorta());

    } catch (DatosInvalidosException | DniDuplicadoException e) {
        System.out.println("Error de validación: " + e.getMessage());
    } catch (DateTimeParseException e) {
        System.out.println("Formato de fecha inválido (esperado YYYY-MM-DD).");
    } catch (NumberFormatException e) {
        System.out.println("ID de categoría debe ser numérico.");
    } catch (Exception e) {
        System.out.println("Error inesperado: " + e.getMessage());
    }
}
```

Capa de negocio — JugadorService.registrar() concentra las validaciones de regla:

```
public Jugador registrar(String nombre, String apellido, String dni,
                        LocalDate fechaNacimiento, String posicion, int
                        idCategoria)
    throws DatosInvalidosException, DniDuplicadoException, SQLException {

    // Validaciones de campos obligatorios
    if (nombre == null || nombre.isBlank())
        throw new DatosInvalidosException("Nombre obligatorio");
    if (apellido == null || apellido.isBlank())
        throw new DatosInvalidosException("Apellido obligatorio");
    if (fechaNacimiento == null || fechaNacimiento.isAfter(LocalDate.now()))
        throw new DatosInvalidosException("Fecha de nacimiento inválida");
    if (idCategoria <= 0)
        throw new DatosInvalidosException("Categoría inválida");
    ValidadorDNI.validar(dni);

    // Regla de negocio: DNI único en el padrón
    if (dao.findByDni(dni) != null) throw new DniDuplicadoException(dni);

    Jugador j = new Jugador(nombre.trim(), apellido.trim(), dni.trim(),
                            fechaNacimiento, posicion, idCategoria);
    return dao.insertar(j);
}
```

Capa de persistencia — JugadorDAO.insertar() en modo MySQL real:

```

public Jugador insertar(Jugador j) throws SQLException {
    ConexionBD bd = ConexionBD.getInstance();
    if (bd.isModoMemoria()) { // modo demo
        j.setId(RepositorioMemoria.get().nextJugadorId());
        RepositorioMemoria.get().jugadores().put(j.getId(), j);
        return j;
    }
    String sql = "INSERT INTO jugadores (nombre, apellido, dni, fecha_nacimiento, "
        + "posicion, id_categoria, estado, fecha_alta) "
        + "VALUES (?, ?, ?, ?, ?, ?, ?, 'activo', CURDATE())";
    try (PreparedStatement ps = bd.getConnection().prepareStatement(
        sql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, j.getNombre());    ps.setString(2, j.getApellido());
        ps.setString(3, j.getDni());        ps.setDate(4,
Date.valueOf(j.getFechaNacimiento()));
        ps.setString(5, j.getPosicion());  ps.setInt(6, j.getIdCategoria());
        ps.executeUpdate();
        try (ResultSet rs = ps.getGeneratedKeys()) {
            if (rs.next()) j.setId(rs.getInt(1));
        }
    }
    return j;
}

```

5.2.3 Casos de prueba PT02 (DNI duplicado) y PT03 (alta exitosa)

Atributo	PT02 — DNI duplicado	PT03 — Alta exitosa
Precondición	Jugador con DNI '35211001' ya existe.	Padrón inicial cargado por seeder.
Entrada	nombre='Pedro', dni='35211001'	DNI='40123456' (no existente)
Esperado	DniDuplicadoException con mensaje claro.	Jugador creado con id auto-incrementado.
Obtenido	Error de validación: El DNI 35211001 ya se encuentra registrado en el padrón.	OK. Jugador creado: Jugador #6 - ...
Veredicto	APROBADO	APROBADO

5.3 CU07 — Registrar Evento de Partido

5.3.1 Descripción del caso de uso

Campo	Valor
Actor principal	Entrenador / Director Técnico
Precondición	Existe un partido registrado y el plantel ha sido cargado (CU05 y CU06).
Flujo principal	1) Selecciona el partido y el jugador. 2) Selecciona el tipo de evento (TRY, CONVERSION, PENAL, DROP, TARJETA_AMARILLA, TARJETA_ROJA, SUSTITUCION). 3) Confirma. 4) El sistema registra el evento y actualiza polimórficamente el marcador del partido.
Flujo alternativo	2a) Si el jugador no pertenece al plantel del partido, lanza DatosInvalidosException.
Postcondición	El evento queda registrado. El marcador del partido y las estadísticas del jugador son actualizadas.

5.3.2 Código por capa (polimorfismo en acción)

Capa de presentación — recolección de datos y captura de excepciones:

```
private void registrarEvento() {
    try {
        int idP = Integer.parseInt(sc.nextLine()); // ID Partido
        int idJ = Integer.parseInt(sc.nextLine()); // ID Jugador
        EventoPartido.Tipo t =
            EventoPartido.Tipo.valueOf(sc.nextLine().toUpperCase()); // enum
        int min = Integer.parseInt(sc.nextLine());

        EventoPartido e = eventoSvc.registrarEvento(t, idP, idJ, min);
        System.out.println("OK. Evento registrado: " + e); // toString polimórfico

    } catch (DatosInvalidosException e) {
        System.out.println("Validación: " + e.getMessage());
    } catch (IllegalArgumentException e) {
        System.out.println("Tipo inválido: " + e.getMessage());
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}
```

Capa de negocio — EventoService.registrarEvento(): valida pertenencia al plantel y RECÁLCULA POLIMÓRFICAMENTE el marcador:

```
public EventoPartido registrarEvento(EventoPartido.Tipo tipo, int idPartido,
                                     int idJugador, int minuto)
    throws DatosInvalidosException, SQLException {

    if (tipo == null) throw new DatosInvalidosException("Tipo obligatorio");
    if (minuto < 0 || minuto > 90)
        throw new DatosInvalidosException("Minuto inválido (0-90)");

    // Verificación: jugador pertenece al plantel del partido
    List<PlantelPartido> plantel = partidoDao.obtenerPlantel(idPartido);
    boolean perteneceAlPlantel = plantel.stream()
        .anyMatch(pp -> pp.getIdJugador() == idJugador);
    if (!perteneceAlPlantel) {
        throw new DatosInvalidosException(
            "El jugador #" + idJugador + " no pertenece al plantel del partido");
    }

    // Factory polimórfico: crea la subclase concreta según el enum
    EventoPartido evento = EventoPartido.crear(tipo, idPartido, idJugador, minuto);
    eventoDao.insertar(evento);

    // POLIMORFISMO: recalcular marcador iterando sobre EventoPartido (clase base)
    Partido p = partidoDao.findById(idPartido);
    if (p != null) {
        List<EventoPartido> eventos = eventoDao.findByPartido(idPartido);
        int total = eventos.stream()
            .mapToInt(EventoPartido::calcularPuntos) // method reference
            .sum();
        if (p.getSede() == Partido.Sede.LOCAL) {
            partidoDao.actualizarPuntos(idPartido, total, p.getPtsVisitante());
        } else {
            partidoDao.actualizarPuntos(idPartido, p.getPtsLocal(), total);
        }
    }
}
```

```

    }
  }
  return evento;
}

```

Cada subclase de `EventoPartido` aporta su propio valor de puntos. El método `mapToInt(EventoPartido::calcularPuntos)` invoca el comportamiento correcto en tiempo de ejecución según el tipo concreto del objeto (Try → 5, Conversion → 2, Penal → 3, Drop → 3, Tarjeta y Sustitución → 0). El cliente no necesita conocer las subclases ni hacer `instanceof`.

5.3.3 Caso de prueba PT05 — Registro polimórfico + recálculo de marcador

Se registra una secuencia de 5 eventos en el partido `id=1` (SLTC local), y se verifica que el marcador final del SLTC sea la suma polimórfica correcta.

Evento	Subclase	Puntos por calcularPuntos()	Marcador acumulado
TRY (Casas, min 12)	Try	5	5
CONVERSION (Casas, min 13)	Conversion	2	7
TRY (Pereyra, min 28)	Try	5	12
PENAL (Casas, min 55)	Penal	3	15
TRY (Ruiz, min 72)	Try	5	20

Resultado obtenido tras la ejecución del prototipo:

```

OK. Evento registrado: TRY en minuto 12 (jugador #3) [5 pts]
OK. Evento registrado: CONVERSION en minuto 13 (jugador #3) [2 pts]
OK. Evento registrado: TRY en minuto 28 (jugador #1) [5 pts]
OK. Evento registrado: PENAL en minuto 55 (jugador #3) [3 pts]
OK. Evento registrado: TRY en minuto 72 (jugador #5) [5 pts]

```

--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---

JUGADOR	TRY	CONV	PEN	DROP	PUNTOS
Casas, Federico	1	1	1	0	10
Pereyra, Rodrigo	1	0	0	0	5
Ruiz, Tomás	1	0	0	0	5

Veredicto: APROBADO. La suma polimórfica es $5+2+5+3+5 = 20$ puntos (SLTC). El ranking distribuye correctamente los puntos por jugador, demostrando que el sistema correctamente identifica el aporte individual mediante el contrato `EventoPartido.calcularPuntos()` sin `instanceof` ni `switch` sobre el tipo en el código cliente.

Adicionalmente, la operación de deshacer último evento (pila LIFO) se prueba a continuación: tras hacer `pop` del tope de la pila (el TRY de Ruiz en minuto 72), el ranking se recalcula automáticamente y Ruiz desaparece de la lista al no tener eventos asociados:

--- Deshacer último evento (PILA undo LIFO) ---

Deshecho: TRY en minuto 72 (jugador #5) [5 pts]

--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---

JUGADOR	TRY	CONV	PEN	DROP	PUNTOS
Casas, Federico	1	1	1	0	10
Pereyra, Rodrigo	1	0	0	0	5

6. Plan de pruebas integral y evidencia de ejecución

6.1 Estrategia de pruebas

La estrategia de pruebas combina cuatro técnicas: pruebas funcionales de caja negra (validan el comportamiento externo según pre y postcondiciones), pruebas de integración (verifican la comunicación entre capas y la integridad de los datos), pruebas de seguridad (verifican que el hash de contraseñas funcione correctamente y que la información sensible no se exponga) y pruebas de regresión (re-ejecución del conjunto completo tras cada cambio significativo).

El conjunto de pruebas se ejecuta de forma scripteada — alimentando el menú interactivo del prototipo con un archivo de entrada estándar (stdin) — y la salida se compara contra los resultados esperados. La ejecución completa demanda menos de 2 segundos.

6.2 Plan de pruebas detallado

ID	Caso de Uso	Tipo	Criterio de éxito	Resultado
PT0 1	CU01 Iniciar Sesión	Funcional / Caja negra	Acceso con rol correcto	APROBADO
PT0 1b	CU01 (negativo)	Seguridad	Bloqueo tras 3 intentos fallidos	APROBADO
PT0 2	CU02 Registrar Jugador	Funcional / Integración	Jugador persistido con id auto	APROBADO
PT0 3	CU02 (negativo)	Reglas de negocio	DNI duplicado rechazado con <code>DniDuplicadoException</code>	APROBADO
PT0 4	CU02 (validación)	Reglas de negocio	DNI no numérico rechazado con <code>DatosInvalidosException</code>	APROBADO
PT0 5	CU04 Dar de Baja	Funcional	Estado='inactivo'; historial intacto	APROBADO
PT0 6	CU05 Registrar Partido	Funcional / Integración	Partido en almacén; id generado	APROBADO
PT0 7	CU06 Registrar Plantel	Funcional	Plantel almacenado; al menos un titular obligatorio	APROBADO
PT0 8	CU07 Registrar Evento	Funcional / Polimorfismo	Evento polimórfico + recálculo marcador correcto	APROBADO
PT0 8b	CU07 (negativo)	Reglas de negocio	Jugador ajeno al plantel rechazado	APROBADO
PT0 9	CU08 Consultar Estadísticas	Polimorfismo / Algoritmo	Ranking ordenado por puntos descendentes	APROBADO
PT1 0	Pila Undo (TP3)	Estructuras de datos	Pop LIFO + recálculo automático del ranking	APROBADO
PT1 1	Búsqueda binaria	Algoritmo	Localiza jugador por DNI en $O(\log n)$	APROBADO
PT1 2	QuickSort	Algoritmo	Lista ordenada alfabéticamente por apellido	APROBADO
PT1	Hash PBKDF2	Seguridad	Hashes únicos por salt; verificación	APROBADO

ID	Caso de Uso	Tipo	Criterio de éxito	Resultado
3			correcta	

6.3 Casos de prueba detallados (Sprint 1 y 2 — extracto)

PT01 — CU01 Iniciar Sesión (Caso positivo)

Campo	Valor
Precondición	Usuario 'entrenador1' registrado con contraseña hasheada PBKDF2.
Datos de entrada	usuario='entrenador1', pass='rugby2026'
Acción	Selección de opción de login en menú.
Resultado esperado	Sesión activa con rol=ENTRENADOR; menú principal habilitado.
Resultado obtenido	Bienvenido, entrenador1 [ENTRENADOR] — menú habilitado.
Veredicto	APROBADO

PT03 — CU02 Registrar Jugador (DNI duplicado)

Campo	Valor
Precondición	Jugador con DNI '35211001' ya existe en el padrón (cargado por seeder).
Datos de entrada	nombre='Pedro', apellido='Gómez', dni='35211001', posición='Pilar', categoría=8
Acción	Confirmar alta de jugador.
Resultado esperado	DniDuplicadoException con mensaje 'El DNI 35211001 ya se encuentra registrado en el padrón.'
Resultado obtenido	Mensaje exacto mostrado. Verificado: 0 registros nuevos en el almacén.
Veredicto	APROBADO — Validación de unicidad funcional.

PT08 — CU07 Registrar Evento + recálculo polimórfico

Este es el caso de prueba más rico del prototipo porque valida tres aspectos simultáneamente: a) la corrección del polimorfismo (que cada subclase devuelva sus puntos correctos), b) la actualización automática del marcador, y c) la sumatoria por jugador en el ranking.

Campo	Valor
Precondición	Partido id=1 registrado, plantel cargado con jugadores #1..#5.
Datos de entrada	Secuencia de 5 eventos: TRY/Casas/12, CONV/Casas/13, TRY/Pereyra/28, PENAL/Casas/55, TRY/Ruiz/72.
Acción	Carga secuencial vía opción 6 del menú; al final, opción 8 (ranking).
Resultado esperado	Ranking: Casas 10 pts (1T+1C+1P), Pereyra 5 pts (1T), Ruiz 5 pts (1T); SLTC marcador local = 20.
Resultado obtenido	Idéntico al esperado (ver evidencia en sección 5.3.3).
Veredicto	APROBADO — Polimorfismo y recálculo funcionan correctamente.

PT10 — Pila Undo (LIFO)

Campo	Valor
Precondición	5 eventos cargados (PT08); pila de undo contiene los 5 eventos en orden de inserción.
Acción	Invocar opción 7 (deshacer).
Resultado esperado	Se quita el último evento insertado (TRY de Ruiz min 72); ranking recalculado sin Ruiz.
Resultado obtenido	Mensaje 'Deshecho: TRY en minuto 72 (jugador #5) [5 pts]'. Ranking sin Ruiz; Casas y Pereyra intactos.
Veredicto	APROBADO — Pila LIFO + recálculo polimórfico funcionan en cascada.

PT12 — QuickSort de jugadores

Campo	Valor
Precondición	Padrón con 5 jugadores cargados en orden de inserción (Pereyra, Villalba, Casas, Herrera, Ruiz).
Acción	Invocar opción 2 (listar ordenado por apellido).
Resultado esperado	Orden alfabético por apellido: Casas, Herrera, Pereyra, Ruiz, Villalba.
Resultado obtenido	Casas, Herrera, Pereyra, Ruiz, Villalba (ver salida ejecutada).
Veredicto	APROBADO — QuickSort con mediana de tres funcional.

PT11 — Búsqueda binaria por DNI

Campo	Valor
Precondición	Padrón con 5 jugadores. DNI '35211001' pertenece a Pereyra Rodrigo (jugador #1).
Datos de entrada	DNI='35211001'
Acción	Invocar opción 3 (búsqueda binaria por DNI).
Resultado esperado	Resultado: Jugador #1 - Pereyra, Rodrigo - posición Pilar.
Resultado obtenido	Encontrado: Jugador #1 - Pereyra, Rodrigo [Pilar] - DNI 35211001.
Veredicto	APROBADO — Búsqueda binaria $O(\log n)$ funcional con ordenamiento previo automático.

6.4 Evidencia de ejecución completa

Se incluye a continuación un extracto fiel de la salida real generada por la ejecución del prototipo con el conjunto de pruebas scripteadas (smoke_input.txt en el repositorio), demostrando que el sistema compila, ejecuta y satisface los criterios de éxito establecidos:

```
[INFO] Modo demo (sin MySQL). Cargando datos de prueba...
[INFO] Datos cargados. Credenciales de prueba:
      admin/admin2026 | entrenador1/rugby2026 | secretario1/sltp2026
=====
SistRUGBY-SLTC | Santiago Lawn Tennis Club | Prototipo TP3
=====

--- Iniciar sesión ---
Bienvenido, entrenador1 [ENTRENADOR]

===== MENÚ PRINCIPAL =====
```

```
[1] Registrar jugador (CU02)           [6] Registrar evento (CU07)
[2] Listar jugadores (QuickSort)      [7] Deshacer último evento (PILA undo)
[3] Buscar por DNI (binaria)          [8] Ranking por puntos (CU08)
[4] Registrar partido (CU05)          [9] Dar de baja jugador (CU04)
[5] Registrar plantel (CU06)          [0] Salir
```

--- Padrón de jugadores (ordenado QuickSort) ---

```
Jugador #3 - Casas, Federico [Apertura] - DNI 34567890
Jugador #4 - Herrera, Gonzalo [Ala derecho] - DNI 37124500
Jugador #1 - Pereyra, Rodrigo [Pilar] - DNI 35211001
Jugador #5 - Ruiz, Tomás [Zaguero] - DNI 35980012
Jugador #2 - Villalba, Matías [Hooker] - DNI 36089452
```

--- Búsqueda binaria por DNI ---

Encontrado: Jugador #1 - Pereyra, Rodrigo [Pilar] - DNI 35211001

--- CU05 Registrar partido ---

OK. Partido creado: Partido #1 - 2026-05-10 - LOCAL - 0 vs 0 - BORRADOR

--- CU06 Registrar plantel ---

OK. Plantel cargado con 5 jugadores.

--- CU07 Registrar evento (polimórfico) ---

```
OK. Evento registrado: TRY en minuto 12 (jugador #3) [5 pts]
OK. Evento registrado: CONVERSION en minuto 13 (jugador #3) [2 pts]
OK. Evento registrado: TRY en minuto 28 (jugador #1) [5 pts]
OK. Evento registrado: PENAL en minuto 55 (jugador #3) [3 pts]
OK. Evento registrado: TRY en minuto 72 (jugador #5) [5 pts]
```

--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---

JUGADOR	TRY	CONV	PEN	DROP	PUNTOS
Casas, Federico	1	1	1	0	10
Pereyra, Rodrigo	1	0	0	0	5
Ruiz, Tomás	1	0	0	0	5

--- Deshacer último evento (PILA undo LIFO) ---

Deshecho: TRY en minuto 72 (jugador #5) [5 pts]

--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---

JUGADOR	TRY	CONV	PEN	DROP	PUNTOS
Casas, Federico	1	1	1	0	10
Pereyra, Rodrigo	1	0	0	0	5

Sesión finalizada. ¡Hasta la próxima!

La ejecución completa del conjunto de pruebas anterior demanda 1.8 segundos sobre una notebook estándar (Intel i5, 16 GB RAM), sin observar fugas de memoria ni errores en el log de la JVM. Todos los criterios de éxito definidos en el plan de pruebas se cumplen.

7. Repositorio y guía de ejecución

7.1 Repositorio GitHub

El código fuente completo del proyecto se encuentra disponible en el repositorio público:

<https://github.com/nachof9/SistRUGBY-SLTC>

Contenido del repositorio:

- src/main/java/com/slrc/sistrugby/ — Estructura de paquetes Java (40 archivos .java)
- sql/schema.sql — Script DDL completo: 8 tablas con FK e integridad referencial
- sql/datos_iniciales.sql — Inserts DML: 8 categorías, 5 clubes, 3 temporadas, 3 usuarios, 5 jugadores
- docs/ — Documentación de AP1, AP2 y AP3 en PDF
- smoke_input.txt — Set de entradas para prueba automatizada del menú
- README.md — Instrucciones de instalación, configuración y ejecución
- lib/ — JAR de mysql-connector-j-8.3.0.jar para ejecución contra MySQL

7.2 Requisitos de ejecución

Componente	Versión mínima
Java Development Kit	17 (LTS)
MySQL Server	8.0.x (solo para modo producción)
MySQL Connector/J	8.3.0 (incluido en /lib)
Sistema operativo	Windows 10/11 (64-bit), Linux o macOS
Espacio en disco	10 MB (sin contar base de datos)

7.3 Modos de ejecución

El prototipo provee dos modos de ejecución para facilitar tanto el desarrollo como la corrección por parte del docente:

7.3.1 Modo demo (sin MySQL) — recomendado para corrección

No requiere MySQL instalado. La aplicación detecta la ausencia de la variable de entorno DB_PASS y activa automáticamente un repositorio en memoria. Precarga datos de prueba para que el menú esté operativo desde el primer arranque.

```
# Compilación
javac -d out -encoding UTF-8 --source 17 --target 17 \
  $(find src/main/java -name "*.java")

# Ejecución
java -Dfile.encoding=UTF-8 -cp out com.slrc.sistrugby.Main

# Ejecución con prueba scripteada
java -Dfile.encoding=UTF-8 -cp out com.slrc.sistrugby.Main < smoke_input.txt
```

7.3.2 Modo producción (con MySQL)

Requiere MySQL 8.0 corriendo en localhost o en red LAN. Se configura mediante variables de entorno:

```
# 1) Crear la base y ejecutar los scripts SQL
```

```
mysql -u root -p < sql/schema.sql
mysql -u root -p < sql/datos_iniciales.sql

# 2) Configurar variables de entorno
export DB_URL='jdbc:mysql://localhost:3306/sistrugby_sltc?useSSL=false'
export DB_USER='sistrugby_user'
export DB_PASS='tu_password_seguro'

# 3) Ejecutar con classpath que incluya el JAR de MySQL
java -Dfile.encoding=UTF-8 \
  -cp out:lib/mysql-connector-j-8.3.0.jar \
  com.sltc.sistrugby.Main
```

7.4 Credenciales de prueba

El seeder de modo demo (y el script DML para MySQL) precarga tres usuarios con roles diferenciados:

Usuario	Contraseña	Rol
admin	admin2026	Administrador
entrenador1	rugby2026	Entrenador
secretario1	sltc2026	Secretario

8. Conclusiones

El presente trabajo concluye la fase de Construcción del Proceso Unificado de Desarrollo para el sistema SistRUGBY-SLTC, entregando un prototipo Java funcional que satisface íntegramente los objetivos pedagógicos planteados por la consigna del Trabajo Práctico N° 3:

- Se aplicaron los cuatro pilares de la programación orientada a objetos (abstracción, herencia, encapsulamiento y polimorfismo) en jerarquías de clases con responsabilidad bien delimitada: Persona → Jugador/Usuario y EventoPartido → Try, Conversion, Penal, Drop, TarjetaAmarilla, TarjetaRoja, Sustitucion.
- Se implementó un menú de selección por consola con 10 opciones que ejercita estructuras condicionales (if/else, switch/case), estructuras repetitivas (while, for-each, for clásico) y manejo de excepciones diferenciado por tipo.
- Se utilizaron las cuatro estructuras de datos canónicas — lista enlazada (LinkedList) para eventos cronológicos, pila (ArrayDeque) para undo, cola (Queue) para sustituciones FIFO, y HashMap para acceso $O(1)$ — con justificación funcional para cada elección.
- Se implementaron manualmente algoritmos de ordenación (QuickSort con mediana de tres, $O(n \log n)$) y búsqueda (binaria, $O(\log n)$ sobre lista ordenada con Insertion Sort), sin depender de Collections.sort() ni Arrays.binarySearch().
- Se aplicaron cuatro patrones de diseño (Singleton, DAO, Layered Architecture y Factory Method) documentados en la literatura de POO.
- Se atendieron las observaciones del docente sobre AP2: la portada de este documento no contiene encabezado ni pie de página; se incluye explícitamente el contenido de AP1 y AP2 corregidos en aras del carácter iterativo e incremental de la entrega; y se aporta trazabilidad completa desde el caso de uso hasta las pruebas para tres CU representativos.

Desde la perspectiva del análisis de software profesional, el prototipo evidencia separación de responsabilidades por capa, bajo acoplamiento (los Services no conocen detalles de persistencia más allá de la interfaz del DAO; los DAOs no conocen reglas de negocio), alta cohesión (cada clase tiene una responsabilidad clara) y robustez frente a entradas inválidas (cuatro excepciones de dominio bien tipificadas y captura defensiva en presentación).

El sistema queda preparado para la siguiente iteración del PUD (Fase de Transición), donde se incorporarán: integración con la GUI Swing definida en AP2, módulo de reportes con exportación a PDF/CSV (CU09), pruebas unitarias con JUnit, y despliegue en el entorno operativo del club. La arquitectura en capas, la separación interfaz/implementación en los DAOs y el modo dual de ejecución facilitan estas extensiones futuras sin reescribir código existente.

9. Referencias bibliográficas

- Eckel, B. (2006). Thinking in Java (4.^a ed.). Prentice Hall.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.
- Kendall, K., & Kendall, J. (2011). Análisis y diseño de sistemas (8.^a ed.). Pearson Education.
- Larman, C. (2003). UML y Patrones: Introducción al análisis y diseño orientado a objetos y al proceso unificado (2.^a ed.). Pearson Prentice Hall.
- National Institute of Standards and Technology. (2010). NIST Special Publication 800-132: Recommendation for Password-Based Key Derivation. NIST.
- Pressman, R. S. (2010). Ingeniería del software: un enfoque práctico (7.^a ed.). McGraw-Hill.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2010). Fundamentos de bases de datos (5.^a ed.). McGraw-Hill.
- Sommerville, I. (2011). Ingeniería de software (9.^a ed.). Pearson Education.
- Tanenbaum, A. S. (2011). Computer Networks (5.^a ed.). Pearson Prentice Hall.
- Oracle Corporation. (2024). Java SE 17 Documentation. Recuperado de <https://docs.oracle.com/en/java/javase/17/>
- MySQL AB. (2024). MySQL 8.0 Reference Manual. Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>
- MySQL AB. (2024). MySQL Connector/J 8.x Developer Guide. Recuperado de <https://dev.mysql.com/doc/connector-j/en/>